The ALC_MOSES manual

version 0.1

Ivan Scivetti

Ada Lovelace Centre, The Science and Technology Facilities Council, United Kingdom

June 2026

About the code

ALC_MOSES is the acronym for [**A**da **L**ovelace **C**entre]_[**MO**delling and **S**ettings for **E**lectromical **S**imulations]. It is a software that generates atomistic models of electrified interfaces automatically, together with input files for Density Functional Theory (DFT) simulations of such interfaces, including the possibility of grand-canonical DFT (GC-DFT). The implemented options also allow the setting of implicit models, which are crucial to account for the electrostatic screening of the solvent (and electrolyte) and reduce the computational cost of MD simulations. In addition, the software can also generate script files for HPC submission. It is important to mention this tool does not execute simulations but set the required input files.

ALC_MOSES is a serial code written in modern Fortran. Its structure for development and maintenance follows the Continuous Integration (CI) practice and it is integrated within the STFC-GitLab DevOps. The code is open-access under license conditions.

This aim of this manual is not only limited to provide a reference guidance for the execution of ALC_MOSES, but also to help the user to familiarise with the capabilities of the code. This manual is a living document, subject to modifications and updates. For this reason, users are advised to consult the latest version.

Disclaimer

The ALC does not fully guarantee the code is free of errors and assumes no legal responsibility for any incorrect outcome or loss of data.

Acknowledgments

- ALC for funding.
- Gilberto Teobaldi for project support.
- Ziwei Chai for help in setting up GC-DFT simulation files using CP2K.
- Manuel dos Santos Dias, Brad Ayres and Jacek Dziedzic for support with the ONETEP code.
- Ciaran O'Brien for discussions on fundamental aspects and applications of the GC-DFT formulation.
- Lesley Mansfield for project management support.

Contents

1 Software functionalities

In this section we describe each of the functionalities implemented in ALC_MOSES. To execute the code, the user needs to define directives within the **SETTINGS** file. Hereinafter, and for the sake of clarity, specific fonts will be used in the main text to differentiate between a ***directive*** (with a given *option* or specification) and a **&Block** containing specific information.

1.1 Settings to build atomistic models

Electrochemical interfaces are generally very complex environments. In fact, current experimental resolution is not yet sufficiently accurate to provide atomistic-level details of the structure and dynamics. To complicate the scenario, real electrode surfaces are far from being flat, exhibiting multiple defects and steps (even when exposed to vacuum). As a result, species are not uniformly deposited throughout the electrode, leading to the formation of rough surfaces [1]. Obviously, considering all these factors is practically unfeasible from the atomistic simulation perspective, and further approximations are required to build simplified models that can capture most of the relevant effects. ALC_MOSES offers an automatic tool for such a purpose.

Within the **SETTINGS** file, the user must specify the *build_model* option for the ***Analysis*** directive. Atomistic models are built using the information of the **&species** block to define the species that are part of the model. The structure of this block is described in Table 3. For each species, the user must set the label, the number and the type of variable (either *fixed* or *variable*). It is important to remark that the number of *variable* species refer to the number of such species that the code will include between topmost region of the electrode and the cell boundary. We will return to this point later.

Together with the specification of the **&species** block, species can well be composed of a set of atoms, as it happens with molecular species (e.g. H_2O). Atomistic detail for each species must be provided with the **&species_components** block. The syntax and format of this block is shown in Table 3. All the species defined in **&species**, even atomic species, must be specified in this block.

The electrode model (slab) must be provided with the **INPUT_ELECTRODE** file, located within the **INPUT_GEOM** folder (sec. 2.1.2). Such slab structure is often set to be lowest energy facets of the electrode material under consideration. To relate the atomic tags defined in **&species_components** with the list of atoms defined in the **INPUT_ELECTRODE** file, the user must also define the **&input_composition** block, whose structure is also described in Table 3. It is compulsory that the the information of **&input_composition** block is consistent with the order of the atomic coordinates listed in the **INPUT_ELECTRODE** file, otherwise the code will complain and abort. Moreover, the atomic tags in **&input_composition** must be defined only once, even those tags that do not have atoms in the **INPUT_ELECTRODE** file. This condition is, maybe, the only non-general aspect of the input settings. The format of the **INPUT_ELECTRODE** file must be consistent with the option of the ***input_model_format*** directive. In case ***input_model_format*** is set to *xyz*, the user must also specify the **&input_cell** block. The format of the generated atomistic models is given by the selected option of the ***output_model_format*** directive.

If there are molecular species involved, the user must also provide the corresponding xyz file for the geometry, within the **INPUT_GEOM** folder. The name of the file must be **species-label.xyz**, where "species-label" refers to the tag defined in **&species** for that particular molecular species. Each file must contain all the atoms for that particular species, and must comply with the format and syntax of section 2.1.3.

The ***normal_vector*** directive sets the vector perpendicular to the surface model, and must be selected between three possible options, namely *c1*, *c2* and *c3*. Two possible options for arranging the insertion of species (either *random* or *deposited*) are available with the definition of the ***arrangement_added_species*** directive (see sec. 1.2 below for description). The species will be inserted when the distance between each of its constituents atoms and the rest of the atoms in the model is larger than a distance criterion. This criterion must be specified with the ***distance_cutoff*** directive.

In principle, the above directives are sufficient to automatically build the interface models, as long as the input electrode model has sufficient space (perpendicular to the surface) to incorporate the target number of species, the input surface area is large enough in comparison to the added species and the value set for the ***distance_cutoff*** directive is reasonable. Making sure to comply with such requirements is user's responsibility. Still, ALC_MOSES was designed to advise the user in case of problems. By default, the electrode model will be centred and species added to both surfaces, while the code optimises the perpendicular space. Also by default, the generated model will have the same surface dimensions than the input model but contain twice the number of *variable* species defined in the **&species** block. Nevertheless, depending on the interface under consideration and the characteristics of the model to be generated, these default settings can be adjusted with the following directives (see Table **??** for details):

- ***repeat_input_model***
- ***add_species_from***
- ***centre_electrode***
- ***both_surfaces***
- ***optimise_size***
- ***add_extra_space***
- ***rotate_species***
- ***delta_space***

IMPORTANT: If the user sets the ***both_surfaces*** directive to `False`, species will be inserted at one side of the slab. In general, this is not appropriate for implicit solvent models (sec. 1.3.4), as the solvation scheme will be applied to both surfaces of the slab, leading to two chemically inequivalent interfaces within the ssame model.

1.2 Summary of the algorithm to build interface models

Given its complexity, a detailed description of the algorithm to generate atomistic models of interfaces goes beyond the scope of this manual. Nevertheless, the procedure can be summarised with the following steps:

1) reading the **input model** from file **INPUT_ELECTRODE** and checking that the simulation cell is consistent with the geometry. Such structure must contain, at least, those atoms of the species that are defined as "fixed" in **&species**, and must represent a reasonable model for the electrode,

2) identifying the number of species (those defined in the **&species** block) present in the input model and determine the resulting amount,

3) grouping atoms that belong to a given species according to the definition of the **&species_components** block,

4) comparing the number of species of the **input model** with those defined in the **&species** block. This step determines how many units of a given species must be removed/added from/to the **input model**,

5) removing species from the **input model** (if required) to match the number of species defined in the **&species** block,

6) inserting species to the **input model** to match the number set in the **&species** block.

7) building a repetition of the modified **input model** according to directive ***repeat_input_model***. This multiple repetition of the **input model** is referred to as **sample model**. By default ***repeat_input_model*** is set to `1 1 1` along the *c1*, *c2* and *c3* axes.

8) printing the generated **sample model** to file. A summary of each generated model is printed as a table in the **OUTPUT** file. The same table is printed to file **MODEL_SUMMARY** inside the **SIMULATION_FILES** folder. Although this construction could be seen as a duplication of the information printed to **OUTPUT**, having a summary of the model provides a quick way to check the composition.

The region within the simulation cell is represented in a 3D-grid set of spatial points, where each cell vector is discretised using the value of directive ***delta_space*** (set to 0.1 Angstrom by default). Unless the user defines the ***add_species_from*** directive, the code automatically finds the position of the plane above which species must be inserted. During insertion, the centroid of the species is set to coincide with one of these grid points. The species will be inserted when the distance between each of its constituents atoms and the rest of the atoms in the model is larger than the distance criterion specified via directive ***distance_cutoff***.

For molecular species, the code randomly rotates the geometry provided in the xyz file (sec. 2.1.3) unless instructed otherwise (see the ***rotate_species*** directive in Table 3). If the species cannot be inserted, the algorithm chooses another grid point. The process is repeated until all the species are inserted.

The name of the file with the generated model is **SAMPLE.format**, where "format" corresponds to the option defined with the ***output_model_format*** directive: either *vasp*, *xyz*, *onetep*, *castep* or *cp2k*. This file is located within the **SIMULATION_FILES** folder. Within the **RESTART** folder, ALC_MOSES generates two files:

1) the **OUTPUT_BACKUP** file, which is a copy of **OUTPUT** to keep record of the output information.
2) the **RECORD_MODELS** file, containing specific information of the atomistic models (sec. 2.2.5). This file allows the user to only generate the input files for atomistic level simulations (sec. 1.3) and scripts with HPC instructions (sec. 1.4), without the need to recompute the atomistic models.

In ALC_MOSES, there are two different ways available for the arrangement of inserted species. The selected option must be given via the ***arrangement_added_species*** directive. The user can choose between *random* and *deposited*, which are described separately in the following:

"Random" insertion of species (Option *random*)

This way to insert of species is suitable for interfaces containing liquids. ALC_MOSES makes use of random number generators to select one of the grid 3D-grid points inside the simulation cell to be centroid of the species to be inserted. Likewise, random number generators are use to randomly rotate molecular species. Unless the user defines the ***add_species_from*** directive, the code automatically finds the position of the plane above which species must be inserted, and only considers this region to be filled. If the species to insert is large, the input model does not contain sufficient space or the value assigned to ***distance_cutoff*** is relatively large, the number of iterations to insert the species may reach a pre-defined maximum limit. In such cases, the code will abort and inform the user. If the input structure and species are reasonable, the user is advised to rerun the code. In case of a new failure, the value of ***distance_cutoff*** must be reduced.

Arrange spacies as "deposited" (Option *deposited*)

This way is convenient to model absorbed species or model films. As described above the code automatically finds the position of the plane, along ***normal_vector***, above which species must be deposited (unless the ***add_species_from*** directive is defined). The grid points over the plane parallel to the surface are swept from such lower limit. In the case of multiple subspecies, each subspecies is added to the surface one after the another in a loop. For example, for 3 subspecies, the sequence of deposition is 1, 2, 3, 1, 2, 3, ..., etc . Species are deposited when all atoms comply with the distance cutoff criterion, while accounting for periodic boundary conditions (PBCs) imposed by the simulation cell. The surface sweeping is conducted for increasing points of the grid along the axis defined with the ***normal_vector*** directive. Depending on the amount of species, the role of the distance cutoff can play a crucial role. In fact, if many species are added and the selected distance cutoff is small, the resulting model for the interface will constitute a (unrealistically) high-energy configuration. If the distance cutoff is large, the process of depositing the species could leave large voids at the interface. It is user's responsibility to select a sensible value for the cutoff distance This can be conveniently optimised by means of a small set of auxiliary DFT simulations aimed at determining the the cutoff distance leading to the lowest energy for the as prepared systems. In addition, the vacuum region must be sufficiently large to prevent the simulation cell to be filled with species.

1.3 Input files for simulations

The generated atomistic model is not sufficient to start a DFT simulation. In fact, further details are needed to define the interaction between the participating atomic species. This section describes how ALC_MOSES can generate additional input files with directives for simulations of the generated models.

The development of a general infrastructure to automatically generate input files for simulations is challenging due to the following reasons, to name a few:

• Interfaces exhibit very different electronic properties, which in turn require of appropriate settings for meaningful simulations.

• Simulation settings depend on the particular property the user aims to investigate.

• Available codes for simulations have their own particular characteristics and intricacies, not only in the way to compute the interatomic interaction but also in the large amount of different possible options for the execution of simulations.

• Not all computational codes contain the same functionalities.

Relevant to the electrochemical response of materials, we have only considered the generation of input files for **geometry relaxation** and **molecular dynamics (MD)** simulations. The generation of input files is activated with the **&simulation_settings** block within the **SETTINGS** file. The generated files and information depend on the choice of the ***output_model_format*** directive. At the present stage of development, it is only possible to generate files for DFT simulations using the codes VASP [2], CASTEP [3], CP2K [4] and ONETEP [5]. The plan is to progressively account for other computational codes. In case there is a missing specification or an incorrect setting, the code will abort and ask the user to fix the change the settings. In the following sections, only those essential directives to generating input files for simulations will be described. The full set of directives and options for **&simulation_settings** are listed and explained in Table 3.

Inside **&simulation_settings**, ALC_MOSES will first ask for the ***simulation_type*** directive. This should be set to either *relax_geometry* or *MD*. The next compulsory directive is the level of theory to describe the interaction between the atoms, ***theory_level***. To date, the only possible option for this directive is *DFT*. The plan for the future is to implement directives to generate input files using machine learning potentials [6, 7, 8, 9] and tight-binding DFT parameterisations [10].

1.3.1 Settings for DFT input files

Having selected the *DFT* option for ***theory_level***, ALC_MOSES will ask the user to specify the **&DFT_settings** block, which must be closed with directive **&end_DFT_settings**. Inside this sub-block, the user must consider the following points:

• The level for the exchange correlation (XC) approach via directive ***XC_level*** (LDA or GGA) and the XC version using ***XC_version*** (See Table 4 for options). Dispersion corrections can be added with directive ***vdW*** (Table 4).

• To compute the electronic problem, optimised methods for diagonalization are set by default. For all these methods, specification of the smearing method (***smearing***, see Table 4) is compulsory, while the smearing width (***with_smear***) is optional but subject to the selected options.

• For CP2K, the user can also select the Orbital Transformation method with the ***OT*** directive to efficiently compute large systems with band gap. Directive ***smearing*** and ***with_smear*** are not compatible with this setting. In addition, setting the ***gapw*** directive to `True` defines the required instructions for simulations using the Gaussian Augmented Plane Waves (GAPW) method.

• For CASTEP, the user can choose the option *fix_occupancy* for ***smearing*** to fix the occupancy for the bands. This is convenient to compute insulators. For metals or semiconductors, the user can only select options *gaussian* or *fermi*, which set the Density Mixing (DM) scheme by default. The user can select the option *edft* (Ensemble DFT) instead of DM, which instructs CASTEP to perform a self-consistent all-bands wavefunction search.

• For ONETEP, ALC_MOSES assumes by default that the system is an insulator or a wide-gap semiconductor. For metals, the user must select the option *edft* (Ensemble DFT) only with option *fermi* for the ***smearing*** directive. In addition, the user must specify sub-block **&ngwf** to specify the number and radius (in Å) of the NGWFs to describe the orbitals.

• Directives ***energy_cutoff***, ***scf_energy_tolerance*** and ***scf_steps*** are compulsory.

• By default, only the Γ point is considered to sample the Brillouin zone. The user can define a mesh for multiple kpoints using directive ***kpoints***.

• The ***precision*** directive is compulsory only for VASP simulations. Likewise, the user must specify how the number of bands that are treated in parallel using directive ***npar*** [2]. If the Brillouin zone needs to be sampled using multiple k-points, the user can further optimise the parallelisation using directive ***kpar*** [2].

• The pseudopotential for each atomic tag must be specified via the **&pseudo_potentials** sub-block. All pseudo potential files shall be located inside the **DFT/PPs** folder. Only for those codes that use atomic-like basis sets, the user must also specify the name of the basis (see Table 4 for the implemented options of basis sets, via the definition of **&basis_set**). We refer to sec. 2.1.4 and 2.1.5 for the specification of the pseudo potential input files and basis set, respectively. If **&pseudo_potentials** is not defined, ALC_MOSES will not set input files with pseudopotentials, and the generated information will not be sufficient to execute a DFT simulation.

• Directive ***spin_polarised*** must be set for spin polarised simulations. For VASP, specification of directive ***max_l_orbital*** is also required.

• Initial magnetic moments for the atomic sites can be specified with the sub-block **&magnetization**.

• The total magnetization is allowed to change by default. To fix the total magnetization, the user must set the target value with directive ***total_magnetization***. The assigned value must be equal to the initial total magnetic moment, which is computed from the initial magnetic moments and the total number of atoms in the sample model. If there is an inconsistency, ALC_MOSES will complain and abort the execution.

• Inclusion of Hubbard corrections is possible via the specification of the **&hubbard** sub-block, which must specify the corrected l-orbital for each participating atomic species, as well as the value of the Hubbard `U` and exchange `J` terms. If there is any value of `J` different from zero, the anisotropic DFT+(U-J) approach of Anisimov [11] and O'Reagan [12, 13] will be set for VASP and ONETEP, respectively. In contrast, if all values of `J` are zero, the isotropic DFT+U method of Dudarev [14] will be considered for all codes. For CP2K and CASTEP, in contrast, the anisotropic approach is not implemented, and the value of (U-J) will only be treated as isotropic. The **&hubbard** sub-block requires the specification of sub-block **&magnetization**.

Finally, option ***bands*** can be set in case the user needs to improve convergence of the electronic minimisation. In VASP, the specified value will correspond to the total number of bands for the system, whereas for CASTEP and CP2K it corresponds to the inclusion of extra bands.

1.3.2 Ionic degrees of freedom

Block **&motion_settings** is needed to specify how ionic degrees of freedom will be described in the simulation. Directives within this block must be compatible with the option selected for ***simulation_type***. It is compulsory to set the directive ***ion_steps*** to specify the number of ionic steps for the simulation.

If ***simulation_type*** is set to `relax_geometry`, the user must specify directive ***relax_method***. The convergence value for the ionic forces can be set using the ***force_tolerance*** directive. By default, the volume and shape of the simulation cell are kept fixed. Thus, if the user wants to relax the volume and the shape of the cell, directives ***change_cell_volume*** and ***change_cell_shape*** must be set to `True`, respectively. ONETEP does not allow simulations with variable cell. Cell relaxation can be carried out under an external pressure defined by ***pressure***. IMPORTANT: simulation settings are not set to preserve the crystal symmetry. This is because the implemented algorithm to generate atomistic models uses a random number generator to insert and remove species, which inevitably breaks any space group symmetry.

To set a MD simulation, the adopted ensemble and the system temperature must be specified using ***ensemble*** and ***temperature***, respectively. The time step for the simulation is set by default to 1.0 fs, but it

can be modified via directive ***timestep***. This information is sufficient for the NVE ensemble. Specification of extra directives depends on the choice for the ensemble:

• NVT: ***thermostat*** and ***relax_time_thermostat*** are needed. ALC_MOSES only sets thermostats that act over the whole system.

• NPT: the ***thermostat*** and ***pressure*** directives must be defined, as well as ***relax_time_thermostat*** and ***relax_time_barostat***. Only for the CASTEP code, the user must define ***relax_time_barostat***. For VASP and CP2K, the barostat properties are determined from the choice of the thermostat. Both ***change_cell_volume*** and ***change_cell_shape*** must be activated. To date, NPT simulations are not implemented in ONETEP.

• NPH: ***pressure*** and ***relax_time_barostat*** are needed. As for NPT, both ***change_cell_volume*** and ***change_cell_shape*** must be `True`. Specification of directive ***barostat*** is also needed for CASTEP settings. To date, ONETEP does not allow NPH simulations.

In case of species containing isotopes, the user can use the sub-block **&masses** to specify the masses of the involved atomic species. See sec. 1.5.2 for more details.

Note: ALC_MOSES only allows generating input files for geometry optimisation and MD simulations. Any other type of calculations such as band structure, EELS, Time-dependent DFT, Phonons, NMR, non-colinear magnetization, etc, are excluded from ALC_MOSES. Files for single point simulations, even though of little use within the context of ALC_MOSES, can also be set by choosing the option *relax_geometry* for directive ***simulation_type*** and setting the ***ion_steps*** directive to 0.

1.3.3 Grand-Canonical DFT for electrochemical systems

To simulate interfaces under electrochemical conditions, it is required to fix the electrode potential. Consequently, the interface model can freely exchange electrons with the external circuit (electron reservoir). For such systems, one needs to use the Grand-Canonical extension of DFT (GC-DFT), where the relevant quantity to minimise is not the total energy but the grand-potential [15]. To date, generating input files for GC-DFT simulations with ALC_MOSES is only possible for the ONETEP and CP2K codes through the definition of the **&gcdft** sub-block (Table 3). The reader is referred to Refs. [16] and [17] for details of the GC-DFT implementation in ONETEP and CP2K, respectively.

1.3.4 Including implicit solvent

The simulation of solid-liquid interfaces still constitutes a major challenge to computational research. This is not only due to the complexity of the interactions at the interface between two different phases (liquid and solid), but also related to the need of sampling multiple configurations for the liquid constituents to extract meaningful thermodynamic quantities.

To circumvent this inherent limitation, empirical models for describing bulk liquids [18], suitable for solid-liquid interfaces have been implemented with the DFT framework [19] and helped to make important advances towards reliable, yet efficient, computation of electrochemical systems. Such empirical models allow including the effect of the solvent implicitly. We refer the reader to the paper of A. Roldan [20] and references therein for a more detail description of the challenges and advances in this field.

The inclusion of implicit solvent directives requires the definition of the **&solvation** block, which asks the required input directives for the computation of the Poisson equation. To date, ALC_MOSES allows implicit solvent settings only for the ONETEP and the CP2K codes. The implicit solvent is included by defining a smooth dielectric cavity around the explicit atomic system. Within the cavity, the relative permittivity is 1, whereas in the bulk region of the implicit solvent it must be ϵ_{bulk}. Such a transition for the permittivity must be also smooth, so the solution of the electrostatic equation is numerical stable. The value of ϵ_{bulk} must be set with directive ***dielectric_constant***, where the word bulk refers to the region of the solvent far from

any other phase or solute. With the ***dielectric_function*** directive, the user can choose between four different models:

• **Fattebert-Gygi scheme**

With the *Fattebert-Gygi* option, the permittivity is set to depend on the charge density $\rho(\mathbf{r})$ as follows:

$$\epsilon[\rho(\mathbf{r})] = 1 + \frac{\epsilon_{bulk} - 1}{2}\left(1 + \frac{1 - (\rho(\mathbf{r})/\rho_0)^{2\beta}}{1 + (\rho(\mathbf{r})/\rho_0)^{2\beta}}\right) \tag{1}$$

where the spatial dependence of ϵ is subject to the spatial dependence of ρ [21]. Parameters for the exponent β and the density threshold ρ_0 must be defined with directives ***beta*** and ***density_threshold***, respectively.

• **Andresussi scheme**

If the *Andreussi* option is selected, the functional form for ϵ is given by the following expression [19]:

$$\epsilon[\rho(\mathbf{r})] = \begin{cases} 1 & \rho(\mathbf{r}) > \rho_{max} \\ exp[t(\ln \rho(\mathbf{r}))] & \rho_{min} < \rho(\mathbf{r}) < \rho_{max} \\ \epsilon_{bulk} & \rho(\mathbf{r}) < \rho_{min} \end{cases} \tag{2}$$

where the function $t(\ln \rho(\mathbf{r})$ reads

$$t(\ln \rho(\mathbf{r})) = \frac{\ln[\epsilon_{bulk}]}{2\pi}\left[2\pi\frac{(\ln \rho_{max} - \ln \rho(\mathbf{r})}{(\ln \rho_{max} - \ln \rho_{min})} - \sin\left(2\pi\frac{(ln\rho_{max} - \ln \rho(\mathbf{r})}{(ln\rho_{max} - ln\rho_{min})}\right)\right]. \tag{3}$$

The value of ρ_{min} and ρ_{max} must be specified with the ***density_min_threshold*** and ***density_max_threshold*** directives, respectively.

• **Soft-spheres** (only available for ONETEP)

If the *soft_sphere* option is selected, the cavities are defined using atom-centred overlapping spheres. Details for this method can be found in Ref. [22]. In contrast to the previous two methods, the transition region is controlled using the Δ parameter, which must be set with the ***soft_sphere_delta***. By default, the radii for the soft spheres are obtain from the vdW radii and can be scaled using the ***soft_sphere_delta***. Alternatively, the soft sphere radii for each species can be defined with the **soft_sphere_radii** block (see Table 6).

• **SAA Andreussi** (only available for CP2K)

This scheme is selected with the *saa_andreussi* option (SAA stands for Solvent-Aware-Approach), and represents an improvement of the *andreussi* method that eliminates the unphysical effects of the continuum environment penetrating low-density regions at the interface. The details of this method go beyond the scope of this manual. The interested user is referred to Ref. [23] for details.

Extra considerations

Relevant to the computation with implicit solvent is the inclusion of the apolar terms. The user can define surface tension of the solvent (***solvent_surface_tension***) and the solvent pressure (***solvent_pressure***; this is not a physical pressure but a volumetric correction to solvation). Different extra settings and considerations are required depending on the selected code.

CP2K

The user can define the repulsion parameter with the ***solvent_repulsion_parameter*** directive, set to zero by default. In case the *SAA_Andreussi* option is set for the ***dielectric_function*** directive, the **&SAA_ANDREUSSI** block will be automatically added to the CP2K input (see appendix A).

ONETEP

The user is referred to Ref. [24] and ONETEP webpage [24] for a comprehensive review of the implemented methods. The user will be asked to set the ***apolar_terms***. In case the *SASA* option is selected, the ***sasa_definition*** directive must also be defined. To include the cavitation term only, the *only_cavitation* option must be selected, and ALC_MOSES will ask the user to set ***apolar_scaling*** equal to 1.0. Apolar corrections can be omitted by selecting the *None* for ***apolar_terms***. See Table 6 for details.

From Ref. [25], an important parameter for convergence in ONETEP is the width if the Gaussian functions used to smear the charge distribution of the ions. This must be set with the ***smear_ion_width*** directive, and the recommended value is 0.8, which must be given in atomic units.

During the simulation, it is expected that the cavity changes its shape, influenced by the changes in geometry and electronic density. By selecting the *self_consistent* for the ***cavity_model*** directive, the shape of the cavity is allowed to change during the simulation. This is, however, inconvenient from the computational and numerical point of view, as extra terms in the energy gradients are needed. Because these terms tend to vary rapidly over very localised regions of space, their accurate calculation usually requires fine grids [26]. This problem can be circumvented by choosing the *fixed* option for the ***cavity_model*** directive. This setting fixes the shape of the cavity, which is computed from an initial simulation in vacuum.

Finally, relevant to the convergence, the user should consider adjusting the following keywords for the multigrid solver (DL_MG) [27] using the **&extra_directives** block (sec. 1.3.6):

- mg_use_cg
- mg_max_iters_vcycle
- mg_vcyc_smoother_iter_pre
- mg_vcyc_smoother_iter_post
- mg_defco_fd_order
- mg_max_iters_newton

1.3.5 Electrolyte model

Together with the implicit solvation, GC-DFT simulations require the definition of the electrolyte model to compensate the excess charge of the explicit quantum system. In ALC_MOSES, the settings for the electrolyte are defined using the **&electrolyte** block. Since ONETEP and CP2K use completely different methods, two different sub-blocks are required. We refer the user to Table 7 to complement the following descriptions.

CP2K

The scheme to include the electrolyte model in CP2K is explained in Ref. [17]. In ALC_MOSES, the electrolyte must be defined with the **&planar_counter_charge** sub-block. This instructs the setting of two charged planes. Each plane contains a total charge of -Q/2, being Q the net charge of the explicit interface model, expected to change along the MD simulation. Within this block, the ***distance_to_edge*** directive must be defined to indicate the distance from each of these planes to the edge of the simulation cell. If such quantity is set to "X", that means both planes will be separated by 2X. In turn, the distance between each of these planes and the topmost/botton-most explicit atom is given by the value of the ***add_extra_space*** directive (`6 Angstrom` by default).
By construction, the charge distribution in the planes is assumed to have a Gaussian-type distribution along the direction perpendicular to the surface. The width of the distribution must be specified with the ***gaussian_width*** directive.

ONETEP

In contrast to the counter charge planes, the electrolyte in ONETEP is modelled with the Boltzmann ions. The problem is then solved using the Poisson-Boltzmann equation (PBEq), which is coupled with the DFT

problem for the explicit atomic species. Details of the implemented algorithms for the solution of the PBEq in ONETEP can be found in Refs. [24, 28].

This scheme is activated with the **&poisson_boltzmann** block, within which the user must first specify the treatment for the computation of the PBEq with the ***solver*** directive, and opt between the full non-linear solution (*full*) and the linear approximation (*linearised*). To compensate for the addition of extra charge to the system, the user must also define the ***neutral_scheme*** directive. When the explicit quantum system is charged, electrolyte ions with opposite charge will be attracted. To prevent the Boltzmann ions from becoming too close to the solute interface and leading to unphysical concentrations, the ***steric_potential*** directive offers the possibility to introduce a repulsive potential which, depending on the selected option, it requires the definition of the ***steric_isodensity*** and ***steric_smearing*** directives. To help the numerical stability for the convergence of the full PBEq, the user can also define a value for the (exponential) ***capping*** directive.

The temperature for the Boltzmann ions must be set with the ***boltzmann_temperature*** directive. The solvent radius cutoff for each atomic species must be defined with the **&solvent_radii** block. Finally, the model parameters for the electrolyte Boltzmann ions must be specified in the **&boltzmann_ions**. ALC_MOSES is robust enough to help the user to define the settings depending on the selected options.

1.3.6 Specification of extra directives

Although the implemented capabilities aim to offer a general framework to building input files for simulations, they are not yet sufficient to include all the possible available options and features of the codes. To try to amend for this limitation, ALC_MOSES offers the sub-block **&extra_directives** where the user can specify additional directives that are not considered in the implementation.

Even though this block aims to increase the range of applications and functionalities of the generated input files, the user is fully responsible for the correct definition of these extra directives. The code will only check that there is no duplication in the definition of directives. Definition of blocks within **&extra_directives** are not allowed.

This block is very convenient to define specific parallelisation and optimisation directives, as well as instructions to improve the convergence of the calculations and control input/output information. At present this functionality is implemented only for the VASP, CASTEP and ONETEP codes.

1.3.7 Default settings for selected CP2K directives

Settings for **&simulation** are aimed to capture most of the functionalities for geometry relaxation and MD simulations with CP2K. Still, we had to arbitrarily define additional (unavoidable) directives based on our previous experience with the code. Such directives are listed in Appendix A. If any of such directives needs to be changed, the user has to do it manually, as the use of **&extra_directives** (sec. 1.3.6) is not allowed for the CP2K format.

1.3.8 Summary description of simulation settings

Following the generation of input files for simulations, ALC_MOSES prints a summary of the simulation settings in the **OUTPUT** file. Depending on the settings, there might be simulations directives that need to be adjusted by the user, either to improve the convergence of the electronic problem, optimise the ionic relaxation or correct the settings for MD simulations. To guide the user with this task, ALC_MOSES prints a message with the following header:

```
************************************************************
Aspects to take into consideration for simulations with XYZ
************************************************************
```

indicating which directives need to be monitored for a better performance and correctness of the simulation. XYZ will be the name of code selected to build the input files. There could be directives that ALC_MOSES fails to define correctly. It is user's responsibility to check that the generated parameters are suitable to the problem under consideration.

1.4 Script files for HPC execution

DFT simulations of the generated interface models (sec. 1.1) necessarily require of parallel computing infrastructures to divide the large computational task in smaller tasks that are executed simultaneously using several CPUs. Such a multi-CPUs infrastructure is known as High-Performance-Computing (HPC) facility. To execute parallel simulations in HPC facilities, however, it is necessary to generate script files that instruct the HPC platform (e.g. SLURM) how to make use of the available CPUs and nodes to distribute the computational task. A node is defined as a set of CPUs.

Generation of HPC script is possible via the specification of **`&HPC_settings`** (see Table 3). In case there is a wrong or missing specification, ALC_MOSES will abort and ask the user to address the problem. Even though ALC_MOSES has been developed and tested using the STFC facility SCARF [29], the definition of settings can be applied to any other HPC facilities. The order for the directives as shown in Table 3 is not mandatory.

The user must provide the name of the HPC machine used for the simulations via directive ***`machine_name`***. Directive ***`platform`*** refers to the job scheduling implemented in the HPC facility, and it is required if the ***`machine_name`*** is different from *SCARF*. Various options are possible for the HPC platform (SLURM, LSF, PSUB, etc), but only SLURM has been implemented to date.

Directive ***`project_name`*** is optional, while ***`job_name`*** is compulsory and used to tag the simulation during the run. The number of processes (or tasks) used for the simulation is defined with ***`number_mpi_tasks`*** and must be carefully set according to the problem to be computed. Relevant to the available HPC architecture, the user must set the number of requested nodes (***`number_nodes`***), as well as the number of CPUs per node (***`CPUs_per_node`***). The choice for ***`parallelism_type`*** (MPI-Only or MPI-OpenMP) depends on how the code was built and compiled. If option MPI-OpenMP is set for ***`parallelism_type`***, the user will be asked to provide the value for ***`threads_per_CPU`***.

To select the HPC partition to job submission, user must specify directive ***`queue`***, together with the maximum allocation time via ***`time_limit`***. To specify ***`time_limit`***, the user must set a list of three integers, which correspond to days, hours and minutes, respectively. To date, the maximum time limit for jobs in SCARF is 7 days. It is users' responsibility to verify the maximum allocation time of the HPC facility.

Directive ***`executable`*** is compulsory and can be provided as a path. If the code needs loading modules before job execution, the user must provide them using sub-block ***`&modules`***. Directive ***`exec_options`*** must be defined if the user needs to specify an option for mpirun (different from -np). Similarly, if the code was compiled with MLK libraries, directive ***`mkl`*** must be set to `True`.

HPC script files will be generated in each of the sub-folders together with the atomistic models and input files for simulations. These files are named as **hpc_script-format.sh**, where, to date, *format* can either be `vasp`, `cp2k`, `castep` or `onetep`, depending on the option for ***`output_model_format`***. A summary of HPC settings is also printed towards the end of the **OUTPUT** for user's reference.

IMPORTANT: Relevant to CPU use and HPC optimisation, the user can also specify the memory requirements with ***`memory_per_CPU`*** (in MegaBytes). Nevertheless, unless the user is fully certain of the memory requirements for the simulation, we strongly recommend to omit the specification of ***`memory_per_CPU`***.

1.5 Extra functionalities and features

1.5.1 Files for simulations and HPC scripts without rebuilding models

Building atomistic models can be significantly more time consuming than generating input files for simulation (sec. 1.3) or scripts files for HPC (sec. 1.4). In fact, depending on the system and the involved species, generating atomistic models can take up to several minutes in contrast to less than a second to derive input files for simulation and HPC. In the previous sections, we have shown that all these files can be obtained at once, as long as the user includes the corresponding blocks in the **SETTINGS** file.

Let us assume the two following hypothetical scenarios, where the user executes ALC_MOSES to obtain atomistic models

1) **without having defined** the required blocks for generating input files for atomistic simulations and/or files for submission of simulations to a HPC machine,

2) **having defined** the blocks to generate input files for atomistic simulations and/or HPC script, but after further analysis the user realises either that simulation or HPC settings (or both) must be adjusted/changed.

Re-running the code with the new simulation/HPC settings via option *build_model* is not convenient, because atomistic models will be rebuilt, and this is exactly what the user wants to avoid. In addition, the new generated models will be different from those generated in the first place, because the insertion and removal of species, as well as the rotation of molecular species, is conducted using a random number generator (sec. 1.1). To generate or update simulation/HPC files and keep the atomistic models, ALC_MOSES offers the option *only_simulation_directives* for ***Analysis***. With this option, the code reads file **RECORD_MODELS**, located in folder **RESTART**, which previously was created when running the code with the *build_model* option. The **RECORD_MODELS** file contains the minimum information for each of the atomistic models in a format that allows ALC_MOSES to characterise them. This information, together with the new specification for the simulation and HPC blocks is sufficient to generate the corresponding files. Following this step, the algorithm corroborates that relevant folders and files exist. By comparison between the new generated files with the information found for each model, the code will inform the user which files remain unchanged and which files have been generated or updated to comply with the new specifications.

1.5.2 Including isotopes for MD simulations

By default, there is no requirement to specify the atomic masses for MD simulations. Nevertheless, if the user wants to consider isotopes in the simulation, ALC_MOSES offers the sub-block **&masses**, within **&motion_settings**, where the user must define the mass (in atomic units) for all the atomic tags.

The assigned atomic masses must be consistent with the masses of the species defined in **&species**, whose atomic composition is set in **&species_components**. For example, assuming the deuterated water D2O species, the corresponding settings should be added to the **&species_components** block:

```
&species_components
⋮
  D2O  2  molecule  1.2
  D H 2 || Ow O 1
⋮
&species_components
```

The masses for these atomic constituents must be defined as follows:

```
&masses
  tags   ...   D   ...   Ow
  values ...  2.0  ...  16.0
&end_masses
```

Note 1: For CASTEP MD simulations with the sub-block **&masses**, all those atomic tags that are not set equal to their chemical element must be defined using the symbol ":". The chemical element must be defined before the ":" and any additional specification after ":". For example, deuterium could be defined as "H:D".
Note 2: This functionality is not yet available in ONETEP.

1.5.3 Additional scripts

ALC_MOSES also offers a set of scripts files located within folder **scripts** of the root directory:

• **clean_data.sh**: in case the user decides to rerun ALC_MOSES, the execution of this script is very convenient to delete all the generated files and folder generated previously without taking the risk to deleting any relevant input file by mistake. This scripts can be copied to the working directory or invoked from the working directory. The user must use the Linux command **sh** to execute this script. For example, if the file has been copied to the working directory, the user must execute "`sh clean_data.sh`".

The following set of template files are also provided:

To generates interface model only	**To generate models and files for simulation**
• **SETTINGS-CASTEP-model_only**	• **SETTINGS-CASTEP-simulation**
• **SETTINGS-CP2K-model_only**	• **SETTINGS-CP2K-simulation**
• **SETTINGS-ONETEP-model_only**	• **SETTINGS-ONETEP-simulation**
• **SETTINGS-VASP-model_only**	• **SETTINGS-VASP-simulation**

The user must remove the symbol "#" symbol in front of the directive and complete the specification. To help on this task, the user can find instructions for directives in Table 3 or follow the settings examples of the tutorials.

1.6 Units convention

Table 1 summarises the units that ALC_MOSES adopts for each relevant input directive, as well as the valid input units.

Table 1: Adopted internal units (and valid units) for relevant input variables

Variable	Adopted internal units	Valid input units
Atomistic models		
distance_cutoff	Angstrom	Angstrom, Bohr
delta_space	Angstrom	Angstrom, Bohr
add_species_from	Angstrom	Angstrom, Bohr
add_extra_space	Angstrom	Angstrom, Bohr
Note: Units for the following directives are transformed internally but adopted as specified. For example, if units for Energy_cutoff are defined as eV, the adopted internal units will be eV. For quantifies defined within blocks, refer to their description for the valid input units.		
DFT		
Energy_cutoff	As specified by the input unit	eV, Ry
Smear_width		eV
SCF_energy_tolerance		eV, Hartree
temperature		K
pressure		kb (or kbar)
force_tolerance		eV Angstrom-1, Hartree Bohr-1
timestep		fs (or fsec)
relax_time_thermostat		fs (or fsec)
relax_time_barostat		fs (or fsec)
GCDFT		
reference_potential	As specified by the input unit	eV
electrode_potential		V
Implicit solvation		
smeared_ion_width	As specified by the input unit	Bohr
solvent_surface_tension		N m-1
solvent_pressure		GPa
solvent_repulsion_parameter		N m-1
Electrolyte		
steric_smearing	As specified by the input unit	Bohr
Boltzmann_temperature		K
distance_to_edge		Angstrom
gaussian_width		Angstrom

2 Input-Output structure

This section describes the content and structure of input and output files. Table 2 summarises the general input/output filing structure for the implemented options of the ***analysis*** directive.

Table 2: Description of the required (input) and the generated (output) files for the two possible types of analysis. Files are listed with bullet points. Folders are reported in bold. Files listed below folders indicates that such files are stored within that particular folder. Input file **SETTINGS** must be located in the working directory. The **OUTPUT** file is generated in the same working directory.

Option for ***Analysis***	Required input files	Output files and folders
build_model (section 1.1)	• **SETTINGS** (2.1.1) **INPUT_GEOM** • **INPUT_ELECTRODE** (2.1.2) • .xyz files (2.1.3) **DFT** • **BASIS_SET** (2.1.5) • vdW kernel (2.1.6) **DFT/PPs** • pseudo-potentials (2.1.4)	• **OUTPUT** (2.2.1) **SIMULATION_FILES** • **SAMPLE.format** (2.2.2) • **MODEL_SUMMARY** (2.2.3) • Files for atomistic simulations (2.2.6) • HPC scripts for simulations (2.2.7) **RESTART** • **OUTPUT_RESTART** (2.2.4) • **RECORD_MODELS** (2.2.5)
only_simulation_files (section 1.5.1)	• **SETTINGS** (2.1.1) **DFT** • **BASIS_SET** (2.1.5) • vdW kernel (2.1.6) **DFT/PPs** • pseudo-potentials (2.1.4) **RESTART** • **RECORD_MODELS** (2.2.5)	• **OUTPUT** (2.2.1) **SIMULATION_FILES** • Files for atomistic simulations (2.2.6) • HPC scripts for simulations (2.2.7)

2.1 Input files

2.1.1 The SETTINGS file

This file contains the directives for building models and setting input files for simulation. This file must be present in the folder where the ALC_MOSES is executed from, otherwise the program will print an error message and abort. Comments can be added using the symbol #. It is recommended to add a descriptive header in the first lines of the file for revision purposes. We report the description of implemented directives and valid blocks in Table 3. Different files will be printed depending on the selected options. The code is flexible enough to recognise directives independently of capitalisation. For example, directive ***Analysis***, ***aNalysIs***, ***anAlySiS***, etc, will all be interpreted as ***analysis***. To complement the description of Table 3, the implemented options for selected DFT directives are shown in Table 4 and 5.
The code checks for the correct syntax and format of the defined directives and blocks. If a problem is found, an error message will be printed in file **OUTPUT** and the execution is aborted. A summary of these directives is printed to file **OUTPUT**. In case there is an inconsistency but the calculation can still proceed, ALC_MOSES prints a warning message. If the required settings are not sufficient to perform the calculation, the program will stop and print an error message instructing the user what to fix.

Table 3: Description of directives to be defined in the **SETTINGS** file

Directive/Block	Format	Description and Specification
Analysis	*option*	Type of analysis to be executed. Options are: • *build_model*: generates an interface model compatible with species defined in the **&species** block. If **&dft_settings** and **&hpc_settings** are defined, input files for DFT simulation and scripts files for HPC submission are also generated. • *only_simulation_directives*: only generates input files for DFT simulation and scripts for HPC submission without the need to recompute interface models.
&species		Block for the specification of the N_s chemical species within the model. Format and syntax: &species number_species N_s $Label_1$ $N0_1$ $Type_1$ $Label_2$ $N0_2$ $Type_2$ ⋮ $Label_{N_s}$ $N0_{N_s}$ $Type_{N_s}$ &end_species number_species: amount of chemical species. Label: species name defined by user (string) N0 : Amount of the species in the model (see 1.1) Type: Type of variable. Either *fixed* or *variable*.
&species_components		Atomistic details for the species defined in **&species**. This block is compulsory to build atomistic models. Format and syntax: &species_components $Label_1$ Nc_1 $class_1$ $maxbond_1$ $Tag_{1,1}$ $Element_{1,1}$ $Stoich_{1,1}$ || ... || $Tag_{Nc_1,1}$ $Element_{Nc_1,1}$ $Stoich_{Nc_1,1}$ $Label_2$ N_{c2} $class_2$ $maxbond_2$ $Tag_{1,2}$ $Element_{1,2}$ $Stoich_{1,2}$ || ... || $Tag_{Nc_2,2}$ $Element_{Nc_2,2}$ $Stoich_{Nc_2,2}$ ⋮ $Label_{N_s}$ Nc_{N_s} $class_{N_s}$ $maxbond_{N_s}$ Tag_{1,N_s} $Element_{1,N_s}$ $Stoich_{1,N_s}$ || ... || $Tag_{Nc_{N_s},N_s}$ $Element_{Nc_{N_s},N_s}$ $Stoich_{Nc_{N_s},N_s}$ &end_species_components $Label_i$: Label of species *i*, as defined in **&species**. Nc_i : Number of atomic components for species *i* (integer). Example: 2 for Water (H and O). $class_i$: class for species *i*. Options: *electrode*, *molecule* or *atom*, depending on the selected option for Type in **&species**. The class of species defined as *fixed* in **&species** must be defined as *electrode*. $maxbond_i$: Maximum intramolecular bonding distance for species *i* (only if class = molecule). ||: denotes a separator between component specification. It can be any string (&, $, etc). $Tag_{i,j}$: Tag for the atomic component *i* of species *j* (String). Maximum is four characters. $Element_{i,j}$: Chemical element for the atomic component *i* of species *j*. $Stoich_{i,j}$: Stoichiometry of the atomic component *i* in the species *j* (integer).

Specification for the atomic components of each species does not need to follow the order of **&species**, as long as all species are defined. Atomic tags must be unequivocally defined. Note that Tag is not necessarily equal to the atomic element. Example: hydrogen tag could set as H+, but Element must be H.

&input_composition For each $\mathrm{Tag}_{i,j}$ defined in **&species_components**, this block must specify number of atoms present in the **INPUT_ELECTRODE** file. This block is compulsory for building models. Format and syntax:

```
&input_composition
  tags      Tag_{1,1}   Tag_{2,1}  ...  Tag_{Nc_1,1}    Tag_{1,2}  ...  ...  Tag_{Nc_Ns,Ns}
  amounts   Nat_{1,1}   Nat_{2,1}  ...  Nat_{Nc_1,1}    Nat_{1,2}  ...  ...  Nat_{Nc_Ns,Ns}
&end_input_composition
```

$\mathrm{Nat}_{i,j}$: Number of $\mathrm{Tag}_{i,j}$ atoms present in file **INPUT_ELECTRODE**.
Specification must include all the $\mathrm{Tag}_{i,j}$ atomic components defined in **&species_components**. The order does NOT need to coincide with the specification of **&species_components** but must be consistent with order of atoms as listed in file **INPUT_ELECTRODE**. If there are no $\mathrm{Tag}_{i,j}$ atoms within the **INPUT_ELECTRODE** file, the user must set the value 0 (zero) for that particular $\mathrm{Tag}_{i,j}$.

input_model_format	*option*	**INPUT_ELECTRODE** file format. Options: *vasp*, *xyz*, *onetep* and *castep*.

&input_cell Specification for the cell vectors of the input model, only needed if the format of **INPUT_ELECTRODE** is XYZ. Format and syntax:

```
&input_cell
  c_{1,1}   c_{1,2}   c_{1,3}
  c_{2,1}   c_{2,2}   c_{2,3}
  c_{3,1}   c_{3,2}   c_{3,3}
&end_input_cell
```

Each $c_{i,j}$ correspond to the j component of cell vector i. IMPORTANT: values must be given in Å.

output_model_format	*option*	Format of the generated atomistic models. Options: *xyz*, *vasp*, *onetep*, *cp2k* and *castep*
normal_vector	*option*	This option must indicate which lattice vectors is perpendicular to the surface. Options: *c1*, *c2* or *c3*.
arrangement_added_species	*option*	Sets the way species will be added to the interface model. Options: *random* and *deposited*
distance_cutoff	Value units	Minimum distance between atoms of different species (1.7 Angstrom is the minimum value). Valid units: Angstrom or Bohr.
repeat_input_model	r1 r2 r3	Defines how many repetitions of the input model, along cell vectors (c_1, c_2, c_3), are set to defined the size of the sample model.
add_species from	Value units	Sets the level along ***normal_vector*** from where species start to be deposited. If not defined, the code identifies the level from the outermost atoms. Valid units: Angstrom or Bohr.
centre_electrode	Logical	(True by default) The algorithm centres the model along the direction perpendicular to the surface.
both_surfaces	Logical	(True by default) Adds species at both sides of the slab.
optimise_size	Logical	(True by default). The code optimises the vacuum region using the value of ***add_extra_space***. Only valid if ***both_surfaces*** is True.
add_extra space	Logical	(6 Angstrom by default). The algorithm uses this value to optimise the extend of the vacuum region at both sides of the surface. Only applicable if ***both_surfaces*** and ***optimise_size*** are True.

rotate_species	`Logical`	(`True` by default) Instructs the code to randomly rotate (or not) molecular species for the process of building the atomistic models.
delta_space	`Real` `units`	(Default is `0.1 Angstrom`) Discretization criterion along each vector of the simulation cell. Valid units: `Angstrom` or `Bohr`.
multiple_input_atoms	`Integer`	This directive must be defined only if the code asks for it. Default value is 10000.
multiple_output_atoms	`Integer`	This directive must be defined only if the code asks for it. Default value is 2.

`&simulation_settings` Block with the set of directives to build input files for simulations. Directives do not need to follow a particular order. Comments must start with `#`.

```
&simulation_settings
 simulation_type       Option
 theory_level          Option
 net_charge            Real
 &DFT_settings (if option DFT is selected for theory_level)
  XC_level              Option
  XC_version            Option
  vdw                   Option
  mixing_scheme         Option
  smearing              Option
  width_smear           Real     Units
  energy_cutoff         Real     Units
  scf_steps             Integer
  scf_energy_tolerance  Real     Units
  bands                 Integer
  kpoints               String   Integer_1   Integer_2   Integer_3
  precision             Option     (Only for VASP)
  npar                  Integer    (Only for VASP)
  kpar                  Integer    (Only for VASP)
  OT                    Logical    (Only for CP2K)
  GAPW                   Logical    (Only for CP2K)
  EDFT                  Logical    (Only for CASTEP and ONETEP)

  &gcdft
   # Only valid for ONETEP
    reference_potential     Real     Units
    electrode_potential     Real     Units
    electron_threshold     Real
   # Only valid for CP2K
    target_workfunction     Real     Units
    mixing_coefficient     Real
  &end_gcdft

  &ngwf (Compulsory only for ONETEP)
    tags      Tag_1,1   Tag_2,1  ...  Tag_Nc1,1   Tag_1,2  ...  ...  Tag_NcNs,Ns
    number    N_1,1     N_2,1    ...  N_Nc1,1     N_1,2    ...  ...  N_NcNs,Ns
    radius    R_1,1     R_2,1    ...  R_Nc1,1     R_1,2    ...  ...  R_NcNs,Ns
  &end_ngwf
```

```
  spin_polarised        Logical
  max_l_orbital         Integer     (Only for VASP)
  total_magnetization   Real
  &magnetization
    tags      Tag_1,1   Tag_2,1  ...  Tag_Nc1,1   Tag_1,2  ...  ...  Tag_NcNs,Ns
    values     μ_1,1     μ_2,1  ...   μ_Nc1,1     μ_1,2  ...  ...   μ_NcNs,Ns
  &end_magnetization

  &hubbard
    tags        Tag_1,1   Tag_2,1  ...  Tag_Nc1,1   Tag_1,2  ...  ...  Tag_NcNs,Ns
    l_orbital    ℒ_1,1     ℒ_2,1  ...   ℒ_Nc1,1     ℒ_1,2  ...  ...   ℒ_NcNs,Ns
    U            U_1,1     U_2,1  ...   U_Nc1,1     U_1,2  ...  ...   U_NcNs,Ns
    J            J_1,1     J_2,1  ...   J_Nc1,1     J_1,2  ...  ...   J_NcNs,Ns
  &end_hubbard

  &pseudo_potentials
    Tag_1,1      PPfilename_1,1
      ⋮             ⋮
    Tag_Nc1,1    PPfilename_Nc1,1
    Tag_1,2      PPfilename_1,2
      ⋮             ⋮
    Tag_NcNs,Ns  PPfilename_NcNs,Ns
  &end_pseudo_potentials

  &basis_set    (Only for CP2K)
    Tag_1,1      ABS_1,1
      ⋮             ⋮
    Tag_Nc1,1    ABS_Nc1,1
    Tag_1,2      ABS_1,2
      ⋮             ⋮
    Tag_NcNs,Ns  ABS_NcNs,Ns
  &end_basis_set
&end_DFT_settings

&solvation (See Table 6)
&end_solvation
&electrolyte (See Table 7)
&end_electrolyte

&motion_settings
  ion_steps            Integer
  #=== geometry relaxation (simulation_type set to relax_geometry)
  relax_method         Option
  force_tolerance      Real     Units
  #=== Molecular dynamics (if simulation_type is set to MD)
  timestep             Real     Units
  ensemble             Option
  temperature          Real     Units
  pressure             Real     Units
```

```
   thermostat           Option
   relax_time_thermostat  Real     Units
   relax_time_barostat    Real     Units
   change_cell_volume    Logical
   change_cell_shape     Logical
  &masses
    tags      Tag_{1,1}   Tag_{2,1}  ...  Tag_{Nc_1,1}   Tag_{1,2}  ...  ...  Tag_{Nc_{N_s},N_s}
    values     m_{1,1}     m_{2,1}  ...    m_{Nc_1,1}     m_{1,2}  ...  ...    m_{Nc_{N_s},N_s}
  &end_masses
 &end_motion_settings

 &extra_directives
   directives_1
   ⋮
   directives_N
 &end_extra_directives

&end_simulation_settings
```

simulation_type: Type of simulation. Valid options for *option*: *relax_geometry* (geometry relaxation) and *MD* (Molecular dynamics)
theory_level: Level of theory to describe the interactions between atoms. To date, *DFT* only.
net_charge: Net charge for the system. If negative, there is an excess of electrons in the system.
XC_level: Level of approximation for the XC. Valid options for *option*: *LDA* and *GGA*.
XC_version: version for the exchange and correlation term. See Table 4 for implemented options.
vdw: Type of van der Waals corrections. See Table 4 for options.
mixing_scheme: Type of mixing for the electronic minimisation. See Table 4 for options.
smearing: Type of smearing for electronic convergence (see Table 4).
width_smear: Width for electronic smearing. `Real` must be positive (default is 0.2 eV). `Units`: `eV`.
energy_cutoff: Energy cutoff. `Real` must be positive. Valid units: `eV` and `Ry`.
scf_steps: Maximum number of steps for electronic convergence.
scf_energy_tolerance: Energy tolerance for electronic convergence. `Real` must be positive (default is 1.0 x 10^{-4} eV). Valid options for `Units`: `eV` and `Hartree`.
bands: In VASP, this flag must be the total number of bands. In CASTEP and CP2K, this number must be the extra bands added to the set of default bands.
kpoints: Information of the reciprocal space. `Option` must be the method to define the mesh, either `MPack` or `Automatic`. The set of three integers specifies the amount of kpoints for each dimension of the Brillouin zone.
precision: Precision for the simulation (only for VASP). Options: *normal*, *single* and *accurate*.
npar: number of bands treated in parallel (only for VASP).
kpar: number of k-points treated in parallel (only for VASP).
OT: activates Orbital Transformation (optional only for CP2K).
GAPW: activates the Gaussian Augmented Plane Waves method (optional only for CP2K. The default method for CP2K is GPW).
EDFT: activates Ensemble-DFT (optional only for ONETEP and CASTEP).
&gcdft: Sub-block to generate instructions for Grand-Canonical DFT simulations.This functionality is ONLY available for ONETEP (requires EDFT) and CP2K. Directives depend on the code:
ONETEP: ***reference_potential*** (`Units`: eV, often set to -4.44 eV), ***electrode_potential*** (`Units`: V) and ***electron_threshold*** (`Real`> 0).
CP2K: ***target_workfunction*** (`Units`: eV) and ***mixing_coefficient*** (`Real`> 0).

&ngwf: Sub-block to specify the number and radius of the NGWF for each participating atomic species. The number of NGWF must be larger than zero. If set to -1, ONETEP will assign default values. IMPORTANT: The radii for the NGWFs must be given in Å.
spin_polarised: Flag to activate spin polarised calculations. Default is `False`.
max_l_orbital: (only for VASP) Maximum l-orbital among all the participating atomic species. Allowed values: 0, 1, 2 and 3 (for s, p, d and f orbitals, respectively).
total_magnetization: sets the total magnetization. `Real` must be given in `Bohr magneton`.
&magnetization: Sub-block to specify the initial magnetization for the participating atomic species. The user must list all the atomic tags of **&input_composition**. Valid units: `Bohr magneton`.
&hubbard: Sub-block to specify the Hubbard corrections applied to specific orbitals of the participating chemical species. The user must list all the atomic tags of **&input_composition**. The orbitals to be corrected must be listed for each atomic tag, following the specification of `l_orbital`. Values for the Hubbard `U` and exchange `J` terms must be specified for each atomic tag in units of eV. If there is any value of `J` different from zero, the anisotropic DFT+(U-J) approach of Anisimov [11] and O'Reagan [12, 13] will be set for VASP and ONETEP, respectively. In contrast, if all values of `J` are zero, the isotropic DFT+U method of Dudarev [14] will be considered for all codes. For CP2K and CASTEP, in contrast, the anisotropic approach is not implemented, and the value of (U-J) will only be treated as isotropic. Block **&hubbard** needs the specification of **&magnetization**.
&pseudo_potentials: Sub-block to specify the name of the pseudo potential files (PPfilename) for each of the participating atomic species. The user must list all the the tags of **&input_composition**. All PPfilename files must be located within folder **DFT/PPs**. If this block is not defined, pseudopotentials files will not be generated.
&basis_set: Only required for CP2K. Sub-block to specify the type of atomic basis set (ABS) adopted for each of the participating chemical species. The user must list all the atomic elements using the tags of **&input_composition**. All ABSs must be within file **BASIS_SET** in folder **DFT**. See Table 4 for the different possible choices of ABS.
ion_steps: Number of ionic steps for the simulation.
relax_method: Method for ionic relaxation. See Table 4 for implemented options. Not all options are available for the implemented codes. ALC_MOSES shall inform the user if the option is not compatible with the choice of the output format.
force_tolerance: Force tolerance for ionic relaxation. `Real` must be positive. Valid options for units: `eV Angstrom-1` (eV/Angstrom) and `Hartree Bohr-1` (Hartree/Bohr).
timestep: Time step for MD simulations. `Real` must be positive (default is 0.1 fs). Units must be in `fs` (or `fsec`).
ensemble: Type of ensemble for the execution of molecular dynamics. See Table 4 for options.
temperature: Temperature for MD simulations. `Real` must be positive. Units must be in `K` (Kelvin).
pressure: Pressure for MD simulations, only needed for `NPT` and `NPH` ensembles. Allowed units: `kb` (kilobars).
thermostat: Thermostat used for MD simulations. See Table 4 for options. Not all options are available in the implemented codes.
relax_time_thermostat: relaxation time for the thermostat.
relax_time_barostat: relaxation time for the barostat (only needed for CASTEP).
change_cell_volume: If set to `True`, the volume of the simulation cell is allowed to change.
change_cell_shape: If set to `True`, this flag allows for changes in the shape of the simulation cell. Default is `False`.
&masses: Sub-block to specify the masses of the participating chemical species. The user must list all the atomic tags of **&input_composition**. Values must be given in atomic units.
&extra_directives: Sub-block to include directives corresponding to functionalities that are not implemented. This block is not available for the CP2K format.

&HPC_settings Block with the set of directives to build script files for HPC submission. Directives do not need to follow any particular order. Comments must start with #.

```
&HPC_settings
 machine_name           String
 platform               String
 project_name           String
 job_name               String
 number_mpi_tasks       Integer
 number_nodes           Integer
 CPUs_per_node          Integer
 memory_per_CPU         Integer
 parallelism_type       String
 threads_per_process    Integer
 queue                  String
 time_limit             Integer1   Integer2  Integer3
 executable             String
 mkl                    Logical
 exec_options           String
 &modules
   module1
   ⋮
   moduleN
 &end_modules
&end_HPC_settings
```

machine_name: Name of HPC facility to run the simulation.
platform: Job scheduling implemented in the HPC machine. To date, only the *SLURM* option is implemented. This directive is not needed if ***machine_name*** is set to SCARF.
project_name: Name of the project. This is optional; ***job_name***: Name/Tag for HPC simulation.
number_mpi_tasks: Number of processes (tasks) for the simulation.
number_nodes: Number of nodes for the simulation.
CPUs_per_node: Number of CPUs per node, which is determined by the computing architecture where the DFT job will be executed. This number is often equal to 16, 20, 24, 32, etc.
memory_per_CPU: Requested memory per CPU, in MegaBytes (MB). This is optional and should only be used if the user is certain about the memory requirements of the code.
parallelism_type: Parallelisation for process distribution (either be *MPI-Only* or *MPI-OpenMP*).
threads_per_process: Number of threads per process. This directive is compulsory if ***parallelism_type*** is set to *MPI-OpenMP*. If Values for the `Integer` must be ≥ 1.
queue: Partition of the HPC facility where the job will be submitted to.
time_limit: Time limit for the simulation. `Integer1`, `Integer2` and `Integer3` correspond to days, hours and minutes, respectively. The user must check beforehand the time limits of the machine and the queue where the simulation will be submitted.
executable: name of the executable to be launched. `String` can be also be set as a path to the executable, including the executable.
mkl: it must be set to `True` if MKL libraries were used for the compilation of the code.
exec_options: String to specify options for running, which must be different from `-np X`.
&modules: Sub-block to include modules that might be required for the execution of the job.

Table 4: Implemented options for ***XC_version*** (LDA and GGA type), ***vdW***, ***ensemble***, ***smearing***, ***thermostat***, ***barostat***, ***basis_set***, ***relax_method*** and ***mixing_scheme***. Listed options are not available for all codes. ALC_MOSES will informs the user if a selected option is not implemented for the requested code. See Table 5 to identify which options are implemented for the different codes.

XC_version (GGA-type)	***XC_version*** (LDA-type)
AM05 (Armiento-Mattsson [30])	*CA* (Ceperley-Alder [31])
PW91 (Perdew-Wang [32])	*PZ* (Perdew-Zunger [33])
PBE (Perdew-Burke-Ernzerhof [34])	*Wigner* (Wigner [35])
RP (revised PBE with Padé Approximation [36])	*VWN* (Vosko-Wilk-Nusair [37, 38])
revPBE [39]	*PW92* [40]
PBEsol [41]	*HL* (Heding-Lundqvist) [42]
BLYP [43, 44]	*PADE* [45]
XLYP [46]	***Ensemble***
WC [47]	NVE (microcanonical)
	NVT (canonical)
vdw	NPT (isothermal-isobaric)
DFT-D2 (Grimme [48])	NPH (isoenthalpic-isobaric)
DFT-D3 (Grimme with zero damping [49])	
DFT-D3-BJ (Grimme with Becke-Jonson damping [50])	***Smearing***
TS (Tkatchenko-Scheffler [51])	*Gaussian*
TSH (Tkatchenko-Scheffler with Hirshfeld partitioning [52])	*Fermi* (Fermi-Dirac distribution)
MBD (Many-body dispersion [53, 54])	*Tetrahedron*
dDsC (Dispersion correction [55])	*Window*
g06 (the same as DFT-D2)	*MP* (Methfessel-Paxton)
OBS (Ortmann, Bechstedt and Schmidt [56])	*Fix_occupancy*
JCHS (Jurečka, Černy, Hobza and Salahub [57])	
The following options generally require kernel files	
vdW-DF [58]	***Basis_set*** (only for CP2K)
optPBE [59]	*SZ* (Single Zeta)
optB88 [59]	*DZ* (Double Zeta)
optB86B [60]	*SZP* (Single Zeta Polarizable)
vdW-DF2 [61]	*DZP* (Double Zeta Polarizable)
vdW-DF2-B86R [62]	*TZP* (Triple Zeta Polarizable)
SCAN+rVV10 [63]	*TZ2P* (Triple Zeta 2-Polarizable)
VV10 [64, 65]	
AVV10S [66]	***relax_method***
	CG (Conjugate Gradient)
thermostat	*QN* (Quasi-Newton)
Andersen	*BFGS* (Broyden-Fletcher-Goldfarb-Shanno)
Nose-Hoover	*LBFGS* (Linear BFGS)
CSVR (Canonical Sampling through Velocity Rescaling)	
Multi-Andersen (Multiple-Andersen)	***barostat*** (CASTEP explicit options)
GLE (Generalised Langevin Equation)	*Andersen-Hoover*
Langevin	*Parrinello-Rahman*
AD_Langevin (Adaptative Langevin)	

mixing_scheme *Linear*, *Kerker* [67], *Broyden*[68], *Broyden-2nd* [69], *Multisecant* [4], *Pulay* [70], *Tchebycheff* [71]. ONETEP specific [5]: *damp_fixpoint*, *LNV*,*dnk_linear*, *ham_linear*,*dnk_pulay*, *ham_pulay*,*dnk_listi*, *ham_listi*, *dnk_listb*, *ham_listb*

Table 5: Availability for the implemented options of Table 4 for the codes VASP, CASTEP, CP2K and ONETEP. Specifications for ***basis_set*** and ***barostat*** are only available for CP2K and CASTEP, respectively, and they are not listed in this table.

Option	VASP	CASTEP	CP2K	ONETEP
XC_version (LDA-type)				
CA [31]	Yes	No	No	No
PZ [33]	Yes	Yes	Yes	Yes
Wigner [35]	Yes	No	Yes	No
VWN [37, 38]	Yes	No	Yes	No
PW92 [40]	No	No	Yes	Yes
HL [42]	Yes	No	Yes	No
PADE [45]	No	No	Yes	No
XC_version (GGA-type)				
AM05 [30]	Yes	No	Yes	No
PW91 [32]	Yes	Yes	No	Yes
PBE [34]	Yes	Yes	Yes	Yes
RP [36]	Yes	No	Yes	Yes
revPBE [39]	Yes	Yes	Yes	Yes
PBEsol [41]	Yes	Yes	Yes	Yes
BLYP [43, 44]	Yes	Yes	Yes	Yes
XLYP [46]	No	No	Yes	Yes
WC [47]	No	Yes	Yes	Yes
vdW (dispersion interactions)				
DFT-D2 [48]	Yes	No	Yes	Yes
DFT-D3 [49]	Yes	No	Yes	No
DFT-D3-BJ [50]	Yes	No	Yes	No
TS [51]	Yes	Yes	No	No
TSH [52]	Yes	No	No	No
MBD [53, 54]	Yes	Yes	No	No
dDsC [55]	Yes	No	No	No
g06 [48]	No	Yes	No	No
OABS [56]	No	Yes	No	No
JCHS [57]	No	Yes	No	No
vdW-DF [58]	Yes	No	Yes	Yes
optPBE [59]	Yes	No	Yes	Yes
optB88 [59]	Yes	No	Yes	Yes
optB86B [60]	Yes	No	Yes	No
vdW-DF2 [61]	Yes	No	Yes	Yes
vdW-DF2-B86R [62]	Yes	No	Yes	No
SCAN+rVV10 [63]	Yes	No	Yes	No
VV10 [64, 65]	No	No	No	Yes
AVV10S [66]	No	No	No	Yes
smearing				
Gaussian	Yes	Yes	No	No
Fermi	Yes	Yes	Yes	Yes
Tetrahedron	Yes	No	No	No
Window	No	No	Yes	No
MP	Yes	No	No	No
Fix_occupancy	No	Yes	No	No

relax_method				
CG	Yes	No	Yes	No
QN	Yes	No	No	No
BFGS	No	Yes	Yes	Yes
LBFGS	No	Yes	Yes	Yes
ensemble				
NVE	Yes	Yes	Yes	Yes
NVT	Yes	Yes	Yes	Yes
NPT	Yes	Yes	Yes	No
NPH	Yes	Yes	Yes	No
thermostat				
Andersen	Yes	No	No	Yes
Multi-Andersen	Yes	No	No	No
Nose-Hoover	Yes	Yes	Yes	Yes
CSVR	No	No	Yes	No
GLE	No	No	Yes	No
Langevin	Yes	Yes	No	Yes
AD_Langevin	No	No	Yes	No
mixig_scheme				
Linear	No	Yes	Yes	No
Kerker	Yes	Yes	Yes	No
Broyden	No	No	Yes	No
Broyden-2nd	Yes	Yes	Yes	No
Multisecant	No	No	Yes	No
Pulay	Yes	Yes	Yes	Yes
Tchebycheff	Yes	No	No	No

See Table 4 for options only specific to ONETEP.

Table 6: Description of directives for the **&solvation** block, which sets specifications for simulations with the implicit solvent. To date, ALC_MOSES only allows the definition of implicit solvents for the ONETEP and CP2K.

```
&solvation
  cavity_model               String
  dielectric_constant        Real
  dielectric_function        String
  solvent_surface_tension    Real Units
  solvent_pressure           Real Units
  # Fattebert-Gygi model
  density_threshold          Real
  beta                       Real
  # Andreussi model
  density_min_threshold      Real
  density_max_threshold      Real
  # Only valid for ONETEP
  smear_ion_width            Real Units
  # Soft sphere model
  &soft_sphere_radii
     Tag_1,1   Tag_2,1  ...  Tag_Nc_1,1   Tag_1,2  ...  ...  Tag_Nc_Ns,N_s
     R_1,1     R_2,1    ...  R_Nc_1,1     R_1,2    ...  ...  R_Nc_Ns,N_s
  &end_soft_sphere_radii
  soft_sphere_delta          Real
  soft_sphere_scale          Real
  # Apolar corrections
  apolar_terms            String
  sasa_definition         String
  apolar_scaling          Real
  # Only relevant to CP2K
  solvent_repulsion_parameter    Real Units
&end_solvation
```

cavity_model: type of cavity for the implicit solvent. Option: *fixed* or *self_consistent*.
dielectric_constant: relative permittivity of the bulk part of the implicit solvent.
dielectric_function: function type to describe the dielectric function. Option: *Fattebert-Gygi*, *Andreussi*, *soft_sphere* (Only valid for ONETEP) or *SAA_Andreussi* (Only vaid for CP2K).
solvent_surface_tension: surface tension for the computation of the apolar terms. Units must be `N m-1`.
solvent_pressure: pressure used to calculate the SAV contribution to the apolar term. Units must be `GPa`.
density_threshold: parameter ρ_0 for the model in atomic units (see Eq. 1).
beta: parameter β for the Fattebert-Gygi model (see Eq. 1).
density_min_threshold: parameter ρ_{min} for the Andreussi model (see Eq. 2).
density_max_threshold: parameter ρ_{max} for the Andreussi model (see Eq. 2).
smear_ion_width: the width of the smeared-ion Gaussians, only in units of `Bohr`. The recommended value is 0.8 Bohr. See Ref. [25].
soft_sphere_radii: This block defines the radii of the involved species for the soft-sphere model [22]. Values must be given in units of Bohr. If this block is undefined, soft-sphere radii will be set to the vdW radii scaled by ***soft_sphere_scale***.
soft_sphere_delta: Controls the steepness of the transition from vacuum to the bulk permittivity.

soft_sphere_scale: Scales the default vdW radii, used to defined the size of the soft spheres. The default is 1.33. This is incompatible with if the radii are defined with the **soft_sphere_radii** block.
apolar_terms: selects how the apolar terms will be considered. Implemented options for *option*: *None*, *only_cavitation*, *SASA* and *SAV*.
sasa_definition: defines the method used in the difference method which calculates the solvent accessible surface area (SASA) of the dielectric. Implemented options: *Density* and *Isosurface*.
apolar_scaling: controls the scaling of the apolar term for the solute-solvent dispersion-repulsion corrections.
solvent_repulsion_parameter: coefficient for the computation of the repulsion term. Units must be `N m-1`.

NOTE: in case the *SAA_Andreussi* option is selected for the ***dielectric_function*** directive, the **&SAA_ANDREUSSI** block is automatically added to the CP2K input (see appendix A).

Table 7: Description of directives for the **&electrolyte** block. Two schemes are available: Poisson-Boltzmann (**&poisson_boltzmann**, only available for ONETEP settings) and Planar Counter Charge (**&planar_counter_charge**), only available for CP2K settings).

```
&electrolyte
  Only vaid for ONETEP
  &poisson_boltzmann
   solver                       Option
   neutral_scheme               Option
   steric_potential             Option
   steric_isodensity            Value
   steric_smearing              Value     Units
   boltzmann_temperature        Value     Units
   capping                      Value
   &solvent_radii
    tags      Tag_1   Tag_2  ...  Tag_{Nc_1,1}   Tag_{1,2}  ...  ...  Tag_{Nc_Ns,N_s}
    values    R^sol_{1,1}  R^sol_{2,1}  ...  R^sol_{Nc_1,1}  R^sol_{1,2}  ...  ...  R^sol_{Nc_Ns,N_s}
   &end_solvent_radii
   &boltzmann_ions
    tags         Tag_1   Tag_2  ...  Tag_{N_B}
    charge         q_1     q_2  ...    q_{N_B}
    conc           C_1     C_2  ...    C_{N_B}
    necs_shift     x_1     x_2  ...    x_{N_B}
   &end_boltzmann_ions
  &end_poisson_boltzmann
  Only vaid for CP2K
  &planar_counter_charge
    distance_to_edge          Value     Angstrom
    gaussian_width            Value     Angstrom
  &end_planar_counter_charge
&end_electrolyte

# Only vaid for ONETEP
solver: treatment of the solution of Poisson-Boltzmann equation. Option: full or linearised.
```

neutral_scheme: method for charge neutralisation. Option: *jellium*, *accessible_jellium*, *counterions_auto*, *counterions_auto_linear* or *counterions_fixed*.
steric_potential: type of potential to prevent the unphysical accumulation of Boltzmann ions close to the explicit quantum system. Option: *Hard-core* or *smoothed-Hard-core*.
steric_isovalue: sets the density isovalue used to determine the solute radial cutoff. Value must be in atomic units. This is only relevant if ***steric_potential*** is NOT *None*. Default is 0.003
steric_smearing: smearing width for the smoothed hard-core cutoff. Value must be positive in Bohr. This is only relevant if ***steric_potential*** is *smoothed-hard-core*. Default is `0.4 Bohr`.
boltzmann_temperature: sets the temperature of the Boltzmann ions in Units of `K`.
capping: sets the exponential cap to help numerical stability when ***solver*** is *full*.
&solvent_radii: defines the solvent radial cutoff for each atomic species. Values must be given in units of `Bohr`, and this is the responsibility of the user.
&boltzmann_ions: defines the Boltzmann ions of the electrolyte. Species `tag`, `charge` (in atomic units) and concentration (`conc`, in mol/L) must be provided. If the *counterions_fixed* option is selected for the `neutral_scheme` directive, the NECS_shift parameters are also needed.
Only vaid for CP2K
distance_to_edge: Distance from the edge of the simulation cell to each charged plane.
gaussian_width: Width of the gaussian distribution for the charge at the plane (recommended: 0.5 Angstrom).

2.1.2 The INPUT_ELECTRODE file

This file is needed if the *build_model* option is set for the ***Analysis*** directive, and must contain the input structure for the electrode. This file must be located within the **INPUT_GEOM** folder. Atomic coordinates in this file must be consistent with the specification of **&input_composition**. The format for the **INPUT_ELECTRODE** file must be specified via directive ***input_model_format***. Allowed format options are: *vasp* (POSCAR), *onetep* and *castep* (.geom) and *xyz*.

2.1.3 Input geometry for molecular species

In case there are molecular species that participate in the reaction, the code will request to include files with the geometry definition for such molecular species. These files must be labelled as **species-label.xyz**, where "species-label" is the label defined in **&species** for that particular molecular species. These files must be located inside folder **INPUT_GEOM**.

Let us consider the water molecule, labelled as H2O, for which its atomic species are tagged as Hw and Ow in **&species_components**. The geometry structure of this molecular species must be defined in file **H2O.xyz** as follows

```
3
geometry
O   0.000    0.000   0.0   Ow
H   0.707    0.707   0.0   Hw
H   -0.707   0.707   0.0   Hw
```

The structure corresponds to the XYZ format, but it requires an additional column (last column) for the specification of the atomic tags. Such tags must be consistent with those defined in **&species_components**. The second line can be anything (usually a descriptive comment) but must be present. The order of atomic species is irrelevant.

2.1.4 Pseudopotential files

Pseudopotentials are needed to compute the electronic structure with DFT. Pseudopotential (PP) files for each atomic species must be copied to folder **DFT/PPs**. The name of the PP file for each species must coincide with the specification within sub-block **&pseudo_potentials**. If this sub-block is not defined, PPs are not set.

• **CASTEP**: Valid PP formats are `.recpot` and `.usp` (ultra-soft). If the **&pseudo_potentials** sub-block is not defined, ALC_MOSES will set instructions to execute the on-the-fly generation of PPs.

• **ONETEP**: `.recpot`, `.upf` and `.paw` formats are allowed. If the user wants to use the PAW formulation, "paw: T" must be defined within the **&extra_directives** block.

• **VASP:** PPs must comply with the **POTCAR** format.

• **CP2K:** PPs format must be consistent with those available in the CP2K repository [72].

IMPORTANT: no pseudopotential file is provided with ALC_MOSES. The user is therefore expected to retrieve suitable pseudopotential files as per code-specific licensing conditions.

2.1.5 The BASIS_SET file

Definition for the basis set is only required to build files for simulation with the CP2K code. The basis set for each atomic tag must be defined in sub-block **&basis_set**. All basis sets must be defined in the **BASIS_SET** file, which must be located within folder **DFT**. It might occur that a particular requested basis sets for a given element has not been included in the **BASIS_SET** file or it does not exist within the CP2K repository [72]. In such a case, ALC_MOSES will stop and inform the user. The following steps are recommended:

i) changing the atomic basis set for the element,

ii) changing "XC_level" and "XC_version",

iii) contacting the CP2K forum for assistance at https://groups.google.com/g/cp2k

iv) generating a new basis set for the element (only for expert users).

2.1.6 Kernel files to set vdW corrections

To set vdW corrections as part of input files for DFT calculations, the user might need to include a kernel file, which must be located inside the **DFT** folder. The name of this file depends on the requested vdW correction and the DFT code. It is important to mention that not all the implemented vdW corrections need of kernel files. ALC_MOSES will inform the user if a kernel is needed. See Table 4 for details.

2.2 Output files

2.2.1 The OUTPUT file

This file contains a summary of the requested analysis and will be generated in the same folder where ALC_MOSES is executed. The file starts with an introductory banner and a brief description of the requested analysis. If the *build_model* option for the ***analysis*** directive is selected, a summary of the generated model is given. If the **&simulation_settings** block is defined, a description of the relevant settings is also provided, together with recommendations for the execution of the simulation.

2.2.2 Atomistic models

These files contain the generated interface models. The user is referred to sec. 1.2 for a more detailed explanation of the implemented algorithm. Generated structures will be printed to file **SAMPLE.format** within the **SIMULATION_FILES** folder. The format for the generated models is set with the ***output_model_format*** directive. Options are: *vasp*, *XYZ*, *onetep*, *castep* and *CP2K*.

2.2.3 The MODEL_SUMMARY file

Together with the **SAMPLE.format** file, a summary of the generated model is recorded to file **MODEL_SUMMARY**. This file is a copy of the table printed in the **OUTPUT** file for that particular model. The table contains the amount of units for each species in the model.

2.2.4 The OUTPUT_BACKUP file

If ***Analysis*** is set to *build_model*, the generated **OUTPUT** file is copied **OUTPUT_BACKUP** inside folder **RESTART**. This file is generated to keep record of the information for the generation of atomistic models.

2.2.5 The RECORD_MODELS file

This file is generated if ***Analysis*** is set to *build_model*. This file contains basic specification for each of the generated atomistic models, and allows the user to obtain input files for atomistic simulations and/or files for HPC submission without the need to recompute the atomistic models (sec. 1.5.1 for details). This file is stored within the **RESTART** folder.

2.2.6 Generated input files for simulations

If the ***Analysis*** directive is set to *build_model* various files are generated depending on the format specified by the directive ***output_model_format***. These files contain all the requested information set in **&simulation_settings** and allow the user to perform atomistic-level simulation of the generated model. The files are generated within the **SIMULATION_FILES** folder. Considering each format separately:

• **VASP**: the generated atomistic model in file **SAMPLE.vasp** is copied to file **POSCAR**. The user will find the **POTCAR** file with the pseudopotentials consistent with the atomic species of the **POSCAR** file. This **POTCAR** file is generated via the concatenation of the pseudo potential files for each atomic species, previously copied to folder PP by the user. File **KPOINTS** contains the specification for sampling of the reciprocal space. The **INCAR** file contains the DFT directives as well as for the ion dynamics.
• **CP2K**: the generated **input.cp2k** file contains all the required directives for simulation. File SAMPLE.cp2k with the atomistic structure is read automatically by CP2K from the specification of directive COORD_FILE_NAME within the **input.cp2k** file. The **BASIS_SET** file and the files with PPs are also copied to the **SIMULATION_FILES** folder.
• **CASTEP**: the generated model.param and model.cell files contain all the required directives for simulation. If **&pseudo_potentials** is defined, each of the pseudo potential files in **DFT/PPs** will be copied to the **SIMULATION_FILES** folder. File CI-castep.cell is only for CI purposes: it contains the same information as model.cell without the cell vectors and the atomic coordinates.
• **ONETEP**: the generated model.dat file contains all the required directives for simulation. Each of the pseudo potential files defined in sub-block **&pseudo_potentials** is also be copied to the **SIMULATION_FILES** folder. File CI-onetep.dat is only for CI purposes: it contains the same information as model.dat without the cell vectors and the atomic coordinates.

2.2.7 The HPC script file

This file, named as **hpc_script-format.sh**, contains directives for allocation and execution of HPC jobs, and are generated only if the **&HPC_settings** block is defined (sec. 1.4). Specification for `"format"` depends on the option set for ***output_model_format***.

3 Worked examples

This section describes the implemented functionalities of ALC_MOSES through examples, so the user can familiarise with the input directives and explore the generated information and output files. Files can be found within the **examples** folder. Words after the symbol # are interpreted by ALC_MOSES as comments, which are used here for description purposes.

3.1 Building a model of a metal/aqueous interface

In the **examples/example1** folder, the user will find an example of how to set directives to build an atomistic interface model of a Pt electrode in contact with water, which in turn contains a Na atom as part of the aqueous environment. Within the **SETTINGS** file, building models requires choosing the *build_model* option for the ***Analysis*** directive.

Relevant to building the model is the specification of the species. Such information must be defined with the **&Species** block (Table 3). For this example, the aim is to add 20 water molecules and 1 Na atom to the model, which consequently correspond to *variable* species. Comment lines (starting with #) are allowed within this block. Here, we use a comment (shown in magenta for clarity) to instruct the user with the correct syntax/format for the specification of the species. Each species must be defined with a convenient label (preferably associated with its chemical composition), followed by its target number N0 and type of variable. The block is closed with the directive **&end_Species**.

```
&species
   number_species  3   # Number of species
   # Label         N0  Type
   Pt              48  fixed
   H2O             20  variable
   Na+             1   variable
&end_Species
```

For the *variable* species, the values of N0 corresponds to the number of target species between the topmost electrode surface and the cell edge. Such values are not the total number of *variable* species within the generated model. In this case, ***both_surfaces*** True (which is also the default) instructs the code that 20 H2O molecules and 1 Na atom will be added to both sides of the slab, making a total number of 40 H2O molecules and 2 Na atoms (plus the 48 *fixed* Pt species corresponding to pristine electrode slab).

The **&species_components** block is needed to specify the atomistic details of the species defined in **&species**:

```
&species_components
  #Pt
  Pt   1  electrode
  Pt Pt 1
  #H2O
  H2O  2  molecule   1.2
  Hw H 2 || Ow O 1
  #Na+
  Na+  1  atom
  Na+ Na 1
&species_components
```

Comments within this block (shown in magenta) must start with the symbol #. We refer the user to Table 3 for a general description of this block. For this particular example, the species tagged as Pt was defined as *fixed* in **&species**. This species is composed of 1 atomic components (Pt), and its class must be defined

as *electrode* (i.e. fixed set of species that are part of the metallic electrode). The next line defines the information of each individual atomic component. The label Pt corresponds the element Pt, and contributes with 1 atom to the stoichiometry.

For the H2O species, the settings show the number of atomic components is equal to 2 (H and O), followed by the class of the species, molecule. This is in contrast to the unit Pt which is fixed and part of the electrode structure. The user must also specify the maximum intramolecular bonding length for water. This parameter depends on the geometry of the molecular species under consideration. Here, we set a sensible value of 1.2 Å. In the next line, the tag Hw for hydrogen is set, followed by the atomic element (H) and number of hydrogen atoms within H2O, equal to 2. Specification for oxygen must follow after the separator ||. Here, the Ow tag is set for the oxygen component, and it is followed by the atomic element O and the number of oxygens per water molecule, equal to 1.

Finally, the Na+ species is followed by 1 (only one constituent) and atom, which is the class for this component. In contrast to the case of water, there is obviously no need to specify any bonding distance criteria for atomic species. In the next line, the same tag Na+ is used for the atomic constituent (but it could be anything else), followed by the chemical element Na and the number of atoms for this species, equal to 1. It is important to remark that the order for the specification of the atomic component for species that contain more than one atom is irrelevant. In fact, one could first define Ow O 1 and then Hw H 2 for H2O, for example. Moreover, no bonding distance is needed for the specification of the electrode type of species. Likewise, the order for the definition of the atomic components for each species does not need to follow the order of **&species**, as long as all species are defined.

The input model for the Pt electrode is given with the **INPUT_ELECTRODE** file within the **INPUT_GEOM** folder, and its structure complies with the POSCAR format (the format used for VASP). Consequently, the ***input_model_format*** directive must be set to *vasp*. It consist of a 4 layers slab in the (111) orientation, with a total of 48 Pt atoms within a supercell with PBCs. The first 8 lines of this file read:

```
Ptslab
1.0000000000000000
8.4510000000000005 0.0000000000000000 0.0000000000000000
0.0000000000000000 9.7590000000000003 0.0000000000000000
0.0000000000000000 0.0000000000000000 30.0000000000000000
Pt
48
Cartesian
```

The user is advised to follow the guidelines of Ref. [73] for a detailed explanation of the POSCAR structure. The sixth line lists the order for the atomic elements of the model (only Pt in this case). The number of atoms for each of these elements is provided in line 7 (48 atoms). To relate the tags (defined in **&species_components**) for each of the elements above, the user must define the following block in the **SETTINGS** file:

```
&input_composition
tags    Pt Na+ Ow Hw
amounts 48 0 0 0
&input_composition
```

In this way, ALC_MOSES can identify each atom listed in the **INPUT_ELECTRODE** and relate it to the atomic tag it belongs to. In this particular case, the input model contains one species and the definition is straightforward. It is important that the user provides the atomic coordinates of the **INPUT_ELECTRODE** file consistent with the information of **&input_composition**, otherwise the code will complain and abort. Moreover, the tags in **&input_composition** must be defined only once, even those tags that do not have atoms in the **INPUT_ELECTRODE** file, as it occurs with all atomic tags expect Pt. This condition in the definition of input models is, maybe, the only non-general aspect of the input settings. Since H2O is the only molecular

species, we also need the H2O.xyz file, which specifies the geometry of the H2O species. This file is located in folder **INPUT_GEOM**. If this file is not present or its format is incorrect, the code will abort.

The remaining directives to build the model are summarised below:

`arrangement_added_species` *random*	`H2O` and `Na+` will be inserted randomly between the topmost Pt layer and the cell edge.
`Normal_vector` *c3*	Since the input cell is orthorhombic and the normal vector is parallel to the z-axis, *c3* is the correct setting here.
`Distance_cutoff 2.3 Angstrom`	Minimum distance between atoms of different species (inter-species distance)
`Repeat_input_model 2 2 1`	The output model (sample) will be a 2x2 duplication of the input model in the xy plane.
`both_surfaces True`	the 20 H2O and the Na atom will be arranged at both side of the slab, making a total of 40 H2O and 2 Na within the model.
`centre_electrode True`	the input Pt slab is shifted within the simulation cell. This directive (True by default) centres the model along *c3*.
`optimise_size True`	When `centre_electrode T`, the size of the cell vector *c3* is optimised using the value of `add_extra_space`.
`add_extra_space 6 Angstrom`	This is the amount of space along *c3* between the topmost and lowermost atoms and the cell edge.

The last remaining directive to generate the atomistic model is ***output_model_format***. The user is asked to run this example for the different options for ***output_model_format***. To familiarise with the code, it is also recommended to review the information generated in the **OUTPUT** file. To understand how models are generated, the user should consider changing the settings ***centre_electrode*** and ***both_surfaces***, as well as the *deposited* option for ***arrangement_added_species***.

3.2 ONETEP simulation files

The example within the **examples/example2** folder uses the files of **examples/example1**, with the difference that the **SETTINGS** file contains the **&simulation_settings** block to generate input files for a MD simulation with the code ONETEP, which is the option set for the ***output_model_format***. In the **SETTINGS** file provided, the **&simulation_settings** block is commented to the purpose of helping the user to go through the definition of the directives.

The user is advised to proceed by first uncommenting the lines with **&simulation_settings** and **&end_simulation_settings**, and run the simulation, which will obviously abort. According to the error messages, the user should continue uncommenting the relevant directives, run and fail again, and continue the process until all the required directives are defined and the code is executed successfully. Even though this procedure is tedious, it constitutes a helpful exercise to familiarise with the directives and their settings. In particular, the user is asked to pay attention to the definition of **&ngwf**. The user must obtain the `.upf` pseudopotentials from Pseudo Dojo [74] and copy them to folder **DFT/PPs**. **IMPORTANT**: If the user wants to use the PAW formulation, "paw: T" must be defined within the **&extra_directives** block.
To setup a GC-DFT simulation, the user must uncomment the **&gcdft** block, which in turn requires **&electrolyte** and the **&solvation** blocks. The user is advised to identify the directives for these blocks, make changes according to the description of secs. 1.3.3 and 1.3.4 (and Table 3) and become familiarised with the generated information. For further details, refer to Refs. [16] and [24].

3.3 CP2K simulation files

The example within the **examples/example3** folder uses the files of **examples/example1**, with the difference that the **SETTINGS** file contains the **&simulation_settings** block to generate input files for a MD simulation

with the code CP2K, which is the option set for the ***output_model_format***. In the **SETTINGS** file provided, the **&simulation_settings** block is commented to the purpose of helping the user to go through the definition of the directives.

The user is advised to proceed by first uncommenting the lines with **&simulation_settings** and **&end_simulation_settings**, and run the simulation, which will obviously abort. According to the error messages, the user should continue uncommenting the relevant directives, run and fail again, and continue the process until all the required directives are defined and the code is executed successfully. In particular, the user is asked to pay attention to the definition of **&basis_set**. Required files for PP, basis sets and dftd3 kernel are provided in **DFT/PPs**.

To setup a GC-DFT simulation, the user must uncomment the **&gcdft** block, which in turn requires **&electrolyte** and the **&solvation** blocks. The user is advised to identify the directives for these blocks, make changes according to the description of secs. 1.3.3 and 1.3.4 (and Table 3) and become familiarised with the generated information. For further details, refer to Ref. [17].

3.4 VASP simulation files

The example within the **examples/example4** folder uses the files of **examples/example1**, with the difference that the **SETTINGS** file contains the **&simulation_settings** block to generate input files for a MD simulation with the code VASP, which is the option set for the ***output_model_format***. In the **SETTINGS** file provided, the **&simulation_settings** block is commented to the purpose of helping the user to go through the definition of the directives.

The user is advised to proceed by first uncommenting the lines with **&simulation_settings** and **&end_simulation_settings**, and run the simulation, which will obviously abort. According to the error messages, the user should continue uncommenting the relevant directives, run and fail again, and continue the process until all the required directives are defined and the code is executed successfully. To included the PPs, the user must add the **&pseudo_potentials** block and copy the corresponding pseudopotentials for each atomic tag (in POTCAR format) within the **DFT/PPs** folder.

The automatic setting of files to perform GC-DFT simulations is not yet implemented in ALC_MOSES.

3.5 CASTEP simulation files

The example within the **examples/example5** folder uses the files of **examples/example1**, with the difference that the **SETTINGS** file contains the **&simulation_settings** block to generate input files for a MD simulation with the code CASTEP, which is the option set for the ***output_model_format***. In the **SETTINGS** file provided, the **&simulation_settings** block is commented to the purpose of helping the user to go through the definition of the directives.

The user is advised to proceed by first uncommenting the lines with **&simulation_settings** and **&end_simulation_settings**, and run the simulation, which will obviously abort. According to the error messages, the user should continue uncommenting the relevant directives, run and fail again, and continue the process until all the required directives are defined and the code is executed successfully. No definition of the **pseudo_potentials** block instructs CASTEP to generate PPs on the fly.

The automatic setting of files to perform GC-DFT simulations is not yet implemented in ALC_MOSES.

3.6 Building a model of metal/film interface

This example shows how to build models of coating material (amorphous Al_2O_3) over a supporting metallic substrate, for which the Pt(111) slab model of the previous examples is also used here. Input files

for this example can be found in folder **examples/example6**. Al_2O_3 is represented by two molecular subspecies, namely AlO and AlO_2 as specified in **&species**. In **&species**, AlO and AlO_2 have the target number. This is a compulsory requirement as both subspecies are part of the same species Al_2O_3. From **&species_components**, oxygen atoms are tagged differently depending on the species. The same applies to the Al atoms.

To generate a deposited film, the ***arrangement_added_species*** must be set to *deposited*. Unless directive ***rotate_species*** is set to `.False.` (default is `.True.`) the implemented algorithm rotates randomly each subspecies before they are deposited, subject to distance cutoff condition. This randomness in the orientation of the deposited subspecies is convenient to model amorphous coating, such as in the present case.

The user is asked to run the example and visualise the generated structure. From the examples of previous sections, the user could also generate input files for the simulation of this interface, selecting the preferred output format (code).

3.7 Obtaining scripts for HPC execution

To become familiarised with this functionality, the user is advised to add the **&hpc_settings** block to the **SETTINGS** file to any of the folders and rerun to generate a script file for HPC submission, which will be printed in the **SIMULATION_FILES** folder.

3.8 Simulation files and HPC scripts without recomputing models

The previous examples shown how input files for simulation and HPC scripts can be generated together with the atomistic models, as long as the user defines **&simulations_settings** and **&hpc_settings** in the **SETTINGS** file. Nevertheless, it may happen the user requests to generate i) atomistic models only, omitting the inclusion of these blocks (sec. 3.1) or ii) atomistic models and input files for simulation only (without HPC scripts), or iii) atomistic models and HPC scripts only (without input files for simulations). It might also happen that the user performs the atomistic simulation for one of the generated models with the selected computational code, and decides it is convenient add/change/remove a simulation/HPC directive. If the user wants to preserve the model generated in first instance but amend simulation or/HPC settings, the option for ***Analysis*** must be change to *only_simulation_directives*.

The user can take any of the provided examples and rerun ALC_MOSES with this new option to modify any DFT/motion/ directive, or add the **&hpc_settings** block. The information provided in the **OUTPUT** file will inform the user of the changes.

3.9 Defining the atomic masses for MD simulations

Relevant to MD simulation of interface models is the possibility to assign isotopic masses to selected elements within the system. Taking any of the examples (except ONETEP), the user is asked to change the mass of the water hydrogen to deuterium via the **&masses** block. Please refer to sec. 1.5.2 for further guidance and the template files within the **scripts** folder.

A Defined settings for CP2K directives

The following table reports only for those CP2K directives (in blue) that have been arbitrarily defined for building input files for the simulation of the generated atomistic models. See Ref. [4] for further details.

Directive	Setting	Notes
PRINT_LEVEL	LOW	Determines a "low" level of generated output information during the run
WALLTIME	300000	Maximum execution time for the run is set to 300000 seconds
PLUS_U_METHOD	MULLIKEN	Mulliken population analysis
&OT		
PRECONDITIONER	FULL_SINGLE_INVERSE	Based on H-eS cholesky inversion.
MINIMIZER	DIIS	Direct inversion in the iterative subspace
N_DIIS	7	Number of history vectors
&END OT		
&OUTER_SCF		
MAX_SCF	10	Maximum number of outer loops
&END OUTER_SCF		
SCF_GUESS	RESTART	Initial guess for the wavefunction
&DIAGONALIZATION		
ALGORITHM	STANDARD	Algorithm to be used for diagonalization
&END DIAGONALIZATION		
&MGRID		
NGRIDS	4	Number of multi-grids
&END MGRID		
&POISSON		
POISSON_SOLVER	ANALYTIC/PERIODIC	ANALYTIC for slabs/PERIODIC for bulk
&END POISSON		
PRESSURE_TOLERANCE	1.00E+02	Pressure tolerance for cell optimization
&THERMOSTAT		
REGION	MASSIVE	region attached to the thermostat
&NOSE		
LENGTH	3	Length of the Nose-Hoover chain
YOSHIDA	3	Order of the yoshida integrator
MTS	2	multiple timesteps for the NH chain
&END NOSE		
&END THERMOSTAT		
&QS		
METHOD	GPW	This is the default
If ***gapw*** is set to `True`:		
METHOD	GAPW	Gaussian Augmented Plane Waves
ALPHA0_H	10	Exponent for hard compensation charge
EPS_PGF_ORB	1.0E-8	Precision of the overlap matrix elements
LMAXN1	6	max L number
&END QS		
&SAA_ANDREUSSI (see Ref. [23])		
RHO_MAX	`Value`	Given by ***density_max_threshold***
RHO_MIN	`Value`	Given by ***density_min_threshold***
F0	0.65	
DELTA_ETA	0.02	

DELTA_ZETA	0.5	
ALPHA_ZETA	2.0	
R_SOLV	2.6	
&END SAA_ANDREUSSI		
&MIXING		
METHOD	NEW_PULAY_MIXING	Only valid option for GC-DFT
ALPHA	0.2	
NBUFFER	8	
QKAPPA	0.25	
QK	3	
QM	0.75	
&END MIXING		

B References

[1] Jutae Kim and Gregory Jerkiewicz. Influence of the Surface Roughness of Platinum Electrodes on the Calibration of the Electrochemical Quartz-Crystal Nanobalance. *Analytical Chemistry*, 89(14):7462–7469, Jul 2017.

[2] The VASP Manual. https://www.vasp.at/wiki/index.php/The_VASP_Manual.

[3] CASTEP. http://www.castep.org/.

[4] CP2K Open Source Molecular Dynamics. https://www.cp2k.org.

[5] ONETEP. https://www.onetep.org/.

[6] Albert P. Bartók, Risi Kondor, and Gábor Csányi. On representing chemical environments. *Phys. Rev. B*, 87:184115, May 2013.

[7] Ryosuke Jinnouchi, Jonathan Lahnsteiner, Ferenc Karsai, Georg Kresse, and Menno Bokdam. Phase Transitions of Hybrid Perovskites Simulated by Machine-Learning Force Fields Trained on the Fly with Bayesian Inference. *Phys. Rev. Lett.*, 122:225701, Jun 2019.

[8] Ryosuke Jinnouchi, Ferenc Karsai, and Georg Kresse. On-the-fly machine learning force field generation: Application to melting points. *Phys. Rev. B*, 100:014105, Jul 2019.

[9] Volker L. Deringer, Albert P. Bartók, Noam Bernstein, David M. Wilkins, Michele Ceriotti, and Gábor Csányi. Gaussian Process Regression for Materials and Molecules. *Chemical Reviews*, 121(16):10073–10141, 2021. PMID: 34398616.

[10] B. Hourahine, B. Aradi, V. Blum, F. Bonafé, A. Buccheri, C. Camacho, C. Cevallos, M. Y. Deshaye, T. Dumitrică, A. Dominguez, S. Ehlert, M. Elstner, T. van der Heide, J. Hermann, S. Irle, J. J. Kranz, C. Köhler, T. Kowalczyk, T. Kubař, I. S. Lee, V. Lutsker, R. J. Maurer, S. K. Min, I. Mitchell, C. Negre, T. A. Niehaus, A. M. N. Niklasson, A. J. Page, A. Pecchia, G. Penazzi, M. P. Persson, J. Řezáč, C. G. Sánchez, M. Sternberg, M. Stöhr, F. Stuckenberg, A. Tkatchenko, V. W.-z. Yu, and T. Frauenheim. DFTB+, a software package for efficient approximate density functional theory based atomistic simulations. *The Journal of Chemical Physics*, 152(12):124101, 2020.

[11] V. I. Anisimov, I. V. Solovyev, M. A. Korotin, M. T. Czyżyk, and G. A. Sawatzky. Density-functional theory and NiO photoemission spectra. *Phys. Rev. B*, 48:16929–16934, Dec 1993.

[12] Edward B. Linscott, Daniel J. Cole, Michael C. Payne, and David D. O'Regan. Role of spin in the calculation of Hubbard *U* and Hund's *J* parameters from first principles. *Phys. Rev. B*, 98:235157, Dec 2018.

[13] Okan K. Orhan and David D. O'Regan. First-principles Hubbard *U* and Hund's *J* corrected approximate density functional theory predicts an accurate fundamental gap in rutile and anatase TiO_2. *Phys. Rev. B*, 101:245137, Jun 2020.

[14] S. L. Dudarev, G. A. Botton, S. Y. Savrasov, C. J. Humphreys, and A. P. Sutton. Electron-energy-loss spectra and the structural stability of nickel oxide: An LSDA+U study. *Phys. Rev. B*, 57:1505–1509, Jan 1998.

[15] Ravishankar Sundararaman, William A. Goddard, and Tomas A. Arias. Grand canonical electronic density-functional theory: Algorithms and applications to electrochemistry. *The Journal of Chemical Physics*, 146(11):114104, 2017.

[16] Arihant Bhandari, Chao Peng, Jacek Dziedzic, Lucian Anton, John R. Owen, Denis Kramer, and Chris-Kriton Skylaris. Electrochemistry from first-principles in the grand canonical ensemble. *The Journal of Chemical Physics*, 155(2):024114, 07 2021.

[17] Ziwei Chai and Sandra Luber. Grand canonical ensemble approaches in cp2k for modeling electrochemistry at constant electrode potentials. *Journal of Chemical Theory and Computation*, 20(18):8214–8228, 2024. PMID: 39240723.

[18] Aleksandr V. Marenich, Christopher J. Cramer, and Donald G. Truhlar. Universal Solvation Model Based on Solute Electron Density and on a Continuum Model of the Solvent Defined by the Bulk Dielectric Constant and Atomic Surface Tensions. *The Journal of Physical Chemistry B*, 113(18):6378–6396, 2009. PMID: 19366259.

[19] Oliviero Andreussi, Ismaila Dabo, and Nicola Marzari. Revised self-consistent continuum solvation in electronic-structure calculations. *The Journal of Chemical Physics*, 136(6):064102, 2012.

[20] Alberto Roldan. Frontiers in first principles modelling of electrochemical simulations. *Current Opinion in Electrochemistry*, 10:1–6, 2018.

[21] Damián A. Scherlis, Jean-Luc Fattebert, François Gygi, Matteo Cococcioni, and Nicola Marzari. A unified electrostatic and cavitation model for first-principles molecular dynamics in solution. *The Journal of Chemical Physics*, 124(7):074103, 2006.

[22] Giuseppe Fisicaro, Luigi Genovese, Oliviero Andreussi, Sagarmoy Mandal, Nisanth N. Nair, Nicola Marzari, and Stefan Goedecker. Soft-Sphere Continuum Solvation in Electronic-Structure Calculations. *Journal of Chemical Theory and Computation*, 13(8):3829–3845, 2017. PMID: 28628316.

[23] Oliviero Andreussi, Nicolas Georg Hörmann, Francesco Nattino, Giuseppe Fisicaro, Stefan Goedecker, and Nicola Marzari. Solvent-aware interfaces in continuum solvation. *Journal of Chemical Theory and Computation*, 15(3):1996–2009, 2019.

[24] Jacek Dziedzic, Arihant Bhandari, Lucian Anton, Chao Peng, James C. Womack, Marjan Famili, Denis Kramer, and Chris-Kriton Skylaris. Practical Approach to Large-Scale Electronic Structure Calculations in Electrolyte Solutions via Continuum-Embedded Linear-Scaling Density Functional Theory. *The Journal of Physical Chemistry C*, 124(14):7860–7872, 2020.

[25] Dziedzic, Jacek and Fox, Stephen J. and Fox, Thomas and Tautermann, Christofer S. and Skylaris, Chris-Kriton. Large-scale DFT calculations in implicit solvent: A case study on the T4 lysozyme L99A/M102Q protein. *International Journal of Quantum Chemistry*, 113(6):771–785, 2013.

[26] ONETEP solvent and electrolyte model. https://www.onetep.org/pmwiki/uploads/Main/Documentation/implicit_solvation_v3.pdf.

[27] James C. Womack, Lucian Anton, Jacek Dziedzic, Phil J. Hasnip, Matt I. J. Probert, and Chris-Kriton Skylaris. DL_MG: A Parallel Multigrid Poisson and Poisson Boltzmann Solver for Electronic Structure Calculations in Vacuum and Solution. *Journal of Chemical Theory and Computation*, 14(3):1412–1432, 2018. PMID: 29447447.

[28] Arihant Bhandari, Lucian Anton, Jacek Dziedzic, Chao Peng, Denis Kramer, and Chris-Kriton Skylaris. Electronic structure calculations in electrolyte solutions: Methods for neutralization of extended charged interfaces. *The Journal of Chemical Physics*, 153(12):124101, 2020.

[29] SCARF: Scientific Computing Application Resource for Facilities. https://www.scarf.rl.ac.uk/.

[30] R. Armiento and A. E. Mattsson. Functional designed to include surface effects in self-consistent density functional theory. *Phys. Rev. B*, 72:085108, Aug 2005.

[31] D. M. Ceperley and B. J. Alder. Ground State of the Electron Gas by a Stochastic Method. *Phys. Rev. Lett.*, 45:566–569, Aug 1980.

[32] John P. Perdew, J. A. Chevary, S. H. Vosko, Koblar A. Jackson, Mark R. Pederson, D. J. Singh, and Carlos Fiolhais. Atoms, molecules, solids, and surfaces: Applications of the generalized gradient approximation for exchange and correlation. *Phys. Rev. B*, 46:6671–6687, Sep 1992.

[33] J. P. Perdew and Alex Zunger. Self-interaction correction to density-functional approximations for many-electron systems. *Phys. Rev. B*, 23:5048–5079, May 1981.

[34] John P. Perdew, Kieron Burke, and Matthias Ernzerhof. Generalized Gradient Approximation Made Simple. *Phys. Rev. Lett.*, 77:3865–3868, Oct 1996.

[35] E. Wigner. On the Interaction of Electrons in Metals. *Phys. Rev.*, 46:1002–1011, Dec 1934.

[36] B. Hammer, L. B. Hansen, and J. K. Nørskov. Improved adsorption energetics within density-functional theory using revised Perdew-Burke-Ernzerhof functionals. *Phys. Rev. B*, 59:7413–7421, Mar 1999.

[37] S. H. Vosko, L. Wilk, and M. Nusair. Accurate spin-dependent electron liquid correlation energies for local spin density calculations: a critical analysis. *Canadian Journal of Physics*, 58(8):1200–1211, 1980.

[38] S. H. Vosko and L. Wilk. Influence of an improved local-spin-density correlation-energy functional on the cohesive energy of alkali metals. *Phys. Rev. B*, 22:3812–3815, Oct 1980.

[39] Yingkai Zhang and Weitao Yang. Comment on "Generalized Gradient Approximation Made Simple". *Phys. Rev. Lett.*, 80:890–890, Jan 1998.

[40] John P. Perdew and Yue Wang. Accurate and simple analytic representation of the electron-gas correlation energy. *Phys. Rev. B*, 45:13244–13249, Jun 1992.

[41] Gábor I. Csonka, John P. Perdew, Adrienn Ruzsinszky, Pier H. T. Philipsen, Sébastien Lebègue, Joachim Paier, Oleg A. Vydrov, and János G. Ángyán. Assessing the performance of recent density functionals for bulk solids. *Phys. Rev. B*, 79:155107, Apr 2009.

[42] L Hedin and B I Lundqvist. Explicit local exchange-correlation potentials. *Journal of Physics C: Solid State Physics*, 4(14):2064–2083, oct 1971.

[43] A. D. Becke. Density-functional exchange-energy approximation with correct asymptotic behavior. *Phys. Rev. A*, 38:3098–3100, Sep 1988.

[44] Chengteh Lee, Weitao Yang, and Robert G. Parr. Development of the Colle-Salvetti correlation-energy formula into a functional of the electron density. *Phys. Rev. B*, 37:785–789, Jan 1988.

[45] S. Goedecker, M. Teter, and J. Hutter. Separable dual-space Gaussian pseudopotentials. *Phys. Rev. B*, 54:1703–1710, Jul 1996.

[46] Xin Xu and William A. Goddard. The X3LYP extended density functional for accurate descriptions of nonbond interactions, spin states, and thermochemical properties. *Proceedings of the National Academy of Sciences*, 101(9):2673–2677, 2004.

[47] Zhigang Wu and R. E. Cohen. More accurate generalized gradient approximation for solids. *Phys. Rev. B*, 73:235116, Jun 2006.

[48] Stefan Grimme. Semiempirical GGA-type density functional constructed with a long-range dispersion correction. *Journal of Computational Chemistry*, 27(15):1787–1799, 2006.

[49] Stefan Grimme, Jens Antony, Stephan Ehrlich, and Helge Krieg. A consistent and accurate ab initio parametrization of density functional dispersion correction (DFT-D) for the 94 elements H-Pu. *The Journal of Chemical Physics*, 132(15):154104, 2010.

[50] Stefan Grimme, Stephan Ehrlich, and Lars Goerigk. Effect of the damping function in dispersion corrected density functional theory. *Journal of Computational Chemistry*, 32(7):1456–1465, 2011.

[51] Alexandre Tkatchenko and Matthias Scheffler. Accurate Molecular Van Der Waals Interactions from Ground-State Electron Density and Free-Atom Reference Data. *Phys. Rev. Lett.*, 102:073005, Feb 2009.

[52] Tomáš Bučko, Sébastien Lebègue, Já nos G. Ángyán, and Jürgen Hafner. Extending the applicability of the Tkatchenko-Scheffler dispersion correction via iterative Hirshfeld partitioning. *The Journal of Chemical Physics*, 141(3):034114, 2014.

[53] Alexandre Tkatchenko, Robert A. DiStasio, Roberto Car, and Matthias Scheffler. Accurate and Efficient Method for Many-Body van der Waals Interactions. *Phys. Rev. Lett.*, 108:236402, Jun 2012.

[54] Alberto Ambrosetti, Anthony M. Reilly, Robert A. Distasio, and Alexandre Tkatchenko. Long-range correlation energy calculated from coupled atomic response functions. *The Journal of Chemical Physics*, 140(18):18A508, 2014.

[55] Stephan N. Steinmann and Clemence Corminboeuf. Comprehensive Benchmarking of a Density-Dependent Dispersion Correction. *Journal of Chemical Theory and Computation*, 7(11):3567–3577, 2011.

[56] F. Ortmann, F. Bechstedt, and W. G. Schmidt. Semiempirical van der Waals correction to the density functional description of solids and molecular structures. *Phys. Rev. B*, 73:205101, May 2006.

[57] Petr Jurečka, Jiří Černý, Pavel Hobza, and Dennis R. Salahub. Density functional theory augmented with an empirical dispersion term. Interaction energies and geometries of 80 noncovalent complexes compared with ab initio quantum mechanics calculations. *Journal of Computational Chemistry*, 28(2):555–569, 2007.

[58] M. Dion, H. Rydberg, E. Schröder, D. C. Langreth, and B. I. Lundqvist. Van der Waals Density Functional for General Geometries. *Phys. Rev. Lett.*, 92:246401, Jun 2004.

[59] Jiří Klimeš, David R Bowler, and Angelos Michaelides. Chemical accuracy for the van der Waals density functional. *Journal of Physics: Condensed Matter*, 22(2):022201, dec 2009.

[60] Jiří Klimeš, David R. Bowler, and Angelos Michaelides. Van der Waals density functionals applied to solids. *Phys. Rev. B*, 83:195131, May 2011.

[61] Kyuho Lee, Éamonn D. Murray, Lingzhu Kong, Bengt I. Lundqvist, and David C. Langreth. Higher-accuracy van der Waals density functional. *Phys. Rev. B*, 82:081101, Aug 2010.

[62] Ikutaro Hamada. van der Waals density functional made accurate. *Phys. Rev. B*, 89:121103, Mar 2014.

[63] Haowei Peng, Zeng-Hui Yang, John P. Perdew, and Jianwei Sun. Versatile van der Waals Density Functional Based on a Meta-Generalized Gradient Approximation. *Phys. Rev. X*, 6:041005, Oct 2016.

[64] Oleg A. Vydrov and Troy Van Voorhis. Nonlocal van der Waals density functional: The simpler the better. *The Journal of Chemical Physics*, 133(24):244103, 2010.

[65] Riccardo Sabatini, Tommaso Gorni, and Stefano de Gironcoli. Nonlocal van der Waals density functional made simple and efficient. *Phys. Rev. B*, 87:041108, Jan 2013.

[66] Torbjörn Björkman. van der Waals density functional for solids. *Phys. Rev. B*, 86:165109, Oct 2012.

[67] G. P. Kerker. Efficient iteration scheme for self-consistent pseudopotential calculations. *Phys. Rev. B*, 23:3082–3084, Mar 1981.

[68] C. G. Broyden. A class of methods for solving nonlinear simultaneous equations. *Math. Comp.*, 19:577–593, 1965.

[69] D. D. Johnson. Modified broyden's method for accelerating convergence in self-consistent calculations. *Phys. Rev. B*, 38:12807–12813, Dec 1988.

[70] Péter Pulay. Convergence acceleration of iterative sequences. the case of scf iteration. *Chemical Physics Letters*, 73(2):393–398, 1980.

[71] H Akai and P H Dederichs. A simple improved iteration scheme for electronic structure calculations. *Journal of Physics C: Solid State Physics*, 18(12):2455–2460, apr 1985.

[72] CP2K repository in GitHub. https://github.com/cp2k/cp2k/tree/master/data.

[73] Structure of the input coordinates consistent with the vasp code. https://www.vasp.at/wiki/index.php/POSCAR.

[74] Pseudo Dojo. https://www.pseudo-dojo.org/.